

ZK-STARK：透明・スケーラブル・耐量子計算完全性

Ben-Sasson, Bentov, Horesh, Riabzev (2018)

網羅的注釈訳・数学的仕組み・Aurora／汎用 ZKP 比較・応用

原論文：Scalable, transparent, and post-quantum secure computational integrity

解説書版 2026-07-15

2018年 ZK-STARK の証明パイプライン



情報理論的 IOP (AIR→APR→ALI→FRI) を、ハッシュベースの暗号コンパイラで実装可能な argument にする

図 0-1：本論文の証明パイプライン。AIR、APR、ALI、FRI から実装可能な STARK へ。

本書の射程

ZKP そのものの初歩的定義は最小限に留め、有限体、Reed-Solomon 符号、AIR、APR、ALI、FRI、完全ゼロ知識、知識抽出、Merkle/Fiat-Shamir 変換を中心に解説する。原論文の本文・付録 A～G を章ごとに網羅し、2019 年 Aurora およびその後の汎用 ZKP との設計上の差、実装上の弱点、応用までを統合した。

目次

本書について／読み方

0. エグゼクティブ・サマリー

第 I 部 数学的基盤：有限体、RS 符号、IOP、AIR、APR、ALI、FRI

1. 論文の問題設定と歴史的的位置
2. 二進有限体・Reed-Solomon 符号・低次数拡張
3. IOP、STIK、STARK の形式
4. AIR：計算を局所多項式制約にする
5. APR：実行・メモリ整合性を代数的グラフへ配置する
6. ALI：多数の制約を二つの RS 近接問題へ圧縮する
7. FRI：線形時間 prover と対数時間 verifier
8. 完全ゼロ知識と知識抽出
9. Merkle 木・BCS 変換・通信量

第 II 部 原論文の注釈付き網羅訳

10. 要旨と導入（原論文 pp.1-13）
11. Methods（pp.14-18）
12. STIK/STARK の定義と主定理（pp.19-23）
13. 付録 A：DNA 照合ベンチマーク
14. 付録 B：AIR から ZK-STARK へ
15. 付録 C：APR reduction
16. 付録 D：ALI protocol
17. 付録 E：DNA profile match の AIR
18. 付録 F・G：SHA2 AIR と affine rearrangeable networks

第 III 部 数式を追う詳細解説

19. 小さな実行トレースによる完全例
20. 健全性誤差の分解
21. 知識抽出
22. 複雑度と実装パラメータ
23. 原論文を批判的に読む

第 IV 部 Aurora および他の汎用 ZKP との比較

24. 2018 STARK 対 2019 Aurora
25. Groth16・PLONK・Marlin・Halo2
26. Bulletproofs・Spartan・Nova・HyperNova
27. Ligero・Fractal・Brakedown
28. DEEP-FRI・STIR・WHIR・Binius・Circle STARK
29. 方式選定表

第 V 部 応用と実装

30. Validity rollup
31. zkVM・再帰・ハイブリッド圧縮
32. 秘匿データ監査
33. AI・科学計算・データパイプライン
34. 実装チェックリスト
35. 限界と研究課題

付録 A 記号／付録 B 定理依存関係／付録 C 原論文図表対応／付録 D 参考文献／付録 E 最終評価

ナビゲーション

DOCX では見出し階層が設定されているため、Word や LibreOffice のナビゲーション／アウトラインから各節へ

移動できる。PDF では検索としおり表示を併用する。

本書について

本書は、2018 年の論文 “Scalable, transparent, and post-quantum secure computational integrity” を、原文の論理順序を保ちながら日本語で再構成した注釈付き解説である。逐語訳ではなく、定義・主張・証明の役割・実装上の意味を明示する「技術的な全訳」を採る。原文の各節と付録 A～G をすべて扱い、式番号・定理番号・図番号を可能な限り対応させた。

ZKP の基本概念は既知と仮定する。ただし、この論文特有の用語である STIK、APR、ALI、RPT、FRI、および「透明」「完全にスケラブル」「もっともらしい耐量子性」が何を意味し、どこまでを保証しないかは丁寧に分解する。

読み方

- **最短経路**：第 0 章、図 0-1、第 4～7 章、第 24 章、第 29 章を読む。
- **数学を追う経路**：第 2～8 章から第 14～16 章、第 19～22 章へ進む。
- **原論文を精読する経路**：第 10～18 章を原論文のページ番号と照合する。
- **方式選定の経路**：第 24～29 章と第 34 章を読む。
- **安全性監査の経路**：第 8 章、第 20～23 章、第 34～35 章を読む。

重要な読み分け

原論文が証明する「存在定理」と、2018 年の proof-of-concept が採用した「具体パラメータ」は同一ではない。後者には ALI と FRI の強い blockwise soundness に関する予想 (Conjectures B.17, B.19) が関与する。また、60-bit 想定のプロトタイプ値を、現代の 128-bit 運用値と混同してはならない。

0. エグゼクティブ・サマリー

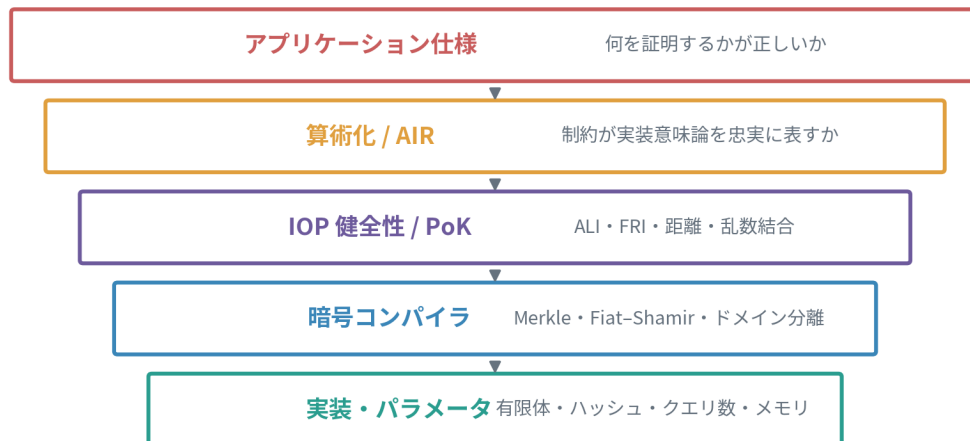
0.1 一言で言えば

本論文は、任意の長い計算を「有限体上の局所制約を満たす低次数符号語」に変換し、その符号語が本当に低次数かを FRI で少数点検査し、Merkle 木と公開乱数で実装可能にすることで、信頼されたセットアップなし・ハッシュ中心・準線形 prover・多対数 verifier を同時に実現した最初期の実用的 ZK-STARK である。

0.2 技術的な新規性

- **AIR**：機械状態の「現在」と「次」を低次数多項式で結び、長い逐次計算を幅の小さな実行トレースにする。
- **APR**：時間順・メモリ順の関係を、アフィン写像で生成されるグラフ上の近傍関係へ埋め込む。
- **register-based encoding**：実行トレース全体を一つの巨大多項式にせず、各レジスタを別々の RS 符号語にして最大次数を大きく下げる。
- **ALI**：多数の AIR/APR 制約をランダム線形結合し、二つの Reed-Solomon 近接性問題へ圧縮する。
- **FRI**：RS 符号への近接性を、prover が線形時間、verifier が対数時間で検査する IOPP として使用する。
- **完全 ZK**：有界独立なランダム低次数拡張と自己還元性を組み合わせ、照会された値の分布を秘密トレースから独立にする。
- **実装最適化**：行単位・coset 単位の Merkle 配置、authentication path の共有、加法 FFT、AES-Davies-Meyer ハッシュを利用する。

STARK の安全性は層ごとに評価する



「透明」「耐量子」という一語だけでは、仕様・算術化・予想依存・ハッシュ仮定・実装欠陥を覆えない。

図 0-2：STARK の安全性を、仕様・算術化・IOP・暗号コンパイラ・実装の層に分けて評価する。

0.3 主定理の意味

空間制限付き計算では、時間 T 、空間 S の計算に対して、prover の有限体演算量は概ね $O(S T \log T)$ 、verifier は $O(\lambda(S + \log T))$ 、証明長は $O(TS)$ field elements となる。 S が $\text{polylog } T$ なら両者ともスケラブルである。一般計算ではメモリ整合性を置換・ルーティングで扱うため、prover は $O(T \log^2 T)$ 、verifier は $O(\lambda \log T)$ 、証明長は $O(T \log T)$ になる。

$$\text{space-bounded: } t_P = O(S T \log T), \quad t_V = O(\lambda(S + \log T))$$

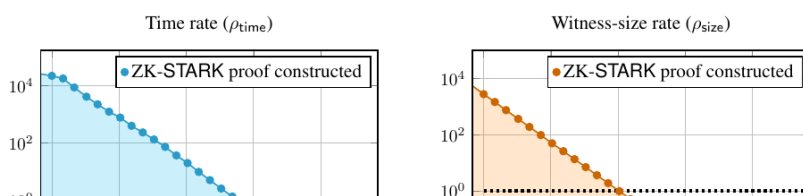
$$\text{general: } t_P = O(T \log^2 T), \quad t_V = O(\lambda \log T)$$

ここで「証明長が線形または準線形」であることと、「最終的に通信する argument が短い」ことは別である。IOP では prover の長い oracle を verifier が少数点しか読まず、暗号コンパイラがその少数点と Merkle 認証路だけを通信させるため、最終 argument が polylogarithmic になる。

0.4 DNA profile match の結果

proof-of-concept は、40-byte の CODIS プロファイルを多数含む秘匿データベースに対し、候補者の DNA プロファイルが完全一致・部分一致・不一致のどれかを、データベースと候補者プロファイルを公開せずに証明する。コミット済みデータへの計算整合性を示すものであり、データベースが現実世界の全データを漏れなく含むことや、入力 DNA の取得過程まで保証するものではない。

指標	論文の値	解釈
プロフィール長	40 bytes	20 loci を 2 値ずつ保持する想定
AIR 幅	81 field elements	逐次機械の状態レジスタ数
サイクル数	62 n	n 件を順番に処理
乗算ゲート換算	90,954 n	他方式との比較用換算値
時間圧縮閾値	約 2.8×10^5 件	これ以上で検証が素朴実行より速い
サイズ圧縮閾値	約 9×10^3 件	これ以上で通信量が witness より小さい
最大生成例	2^{20} 件	約 43 MB のデータベース
同例の比率	時間 1/6、サイズ 1/100	2018 年実装・60-bit 想定



原論文図 1：DPM ベンチマークの時間圧縮率と witness-size 圧縮率。破線 1 未満が圧縮領域。

0.5 本論文の長期的な値

個々の具体パラメータやハッシュ関数、FRI の健全性解析は後続研究で更新されたが、「計算を AIR へ落とし、RS 符号で低次数拡張し、FRI 系 IOPP で検査し、Merkle/Fiat-Shamir で非対話化する」という設計骨格は、現代の STARK 系 zkVM・validity proof・透明な多項式コミットメントへ連続している。Aurora はこの骨格を R1CS に最適化し、DEEP-FRI、STIR、WHIR は低次数検査を改良し、Circle STARK は評価領域の自由度を拡張し、Binius は二進体上の算術化を別方向から洗練した。

結論

この論文の本質は、短い証明そのものよりも、「長い逐次計算を、低次数性と局所整合性だけで検査できる形に変えるコンパイラ」を具体化した点にある。実務で重要なのは STARK という名称ではなく、算術化、符号化、近接性検査、コミットメント、再帰、メモリ管理を一体として設計することである。

第1部 数学的基盤

ZKP の初歩を省き、本論文を読むための代数・符号・IOP 構造に集中する

1. 論文の問題設定と歴史的位置

1.1 計算完全性と秘密データ

論文の中心問題は、秘密データ D に対する計算 $C(D)$ の結果を、データを公開せず、第三者監査人を信頼せず、しかも検証者が D 全体を読み直さずに確認することである。単にコミットメント $\text{cmt}(D)$ を公開しても、コミットメントは「後からデータを差し替えていない」ことしか保証せず、そのデータに対して主張された計算を正しく実行したことは保証しない。

したがって公開主張は、典型的には「公開されたコミットメント cmt と公開出力 y に対し、ある秘密データ D と補助入力 w が存在し、 $\text{cmt}(D)=\text{cmt}$ かつ $C(D,w)=y$ である」という NP/NEXP 型の関係になる。DPM では、データベースと候補者プロファイルの双方のコミットメントを公開し、照合結果だけを公開する。

1.2 六つの設計目標

目標	意味	本論文の実現手段
透明性	秘密の setup trapdoor を持つ主体がいない	公開コイン IOP+ハッシュ
汎用性	任意の効率的計算に適用できる	AIR/PAIR による計算表現
秘匿性	witness 以外を漏らさない	完全 ZK IOP→計算量的 ZK argument
耐量子性	既知の量子攻撃に対してもっともらしく安全	衝突耐性ハッシュと符号理論
知識性	受理させる prover から witness を抽出可能	RS 復号+ALI/APR extractor
完全スケーラビリティ	prover 準線形、verifier 多対数	additive FFT+ALI+FRI

1.3 「透明」と「耐量子」の正確な範

透明性は「何のパラメータも不要」という意味ではない。有限体、評価領域、ハッシュ関数、符号レート、セキュリティパラメータなどの公開パラメータは必要である。問題になるのは、偽証能力を与える秘密 trapdoor を誰かが知り得るかどうかである。STARK では、公開乱数またはランダムオラクルを用いるため、秘密 SRS を必要としない。

耐量子性も無条件の定理ではない。情報理論的 IOP 部分は量子計算能力に依存しないが、実装された argument はハッシュ関数の安全性と Fiat-Shamir 型変換に依存する。Grover 探索はハッシュの preimage 安全性を平方根程度に低下させるため、量子攻撃を考える場合は digest 長・query model・QROM 解析を含めて余裕を取る必要がある。原論文の「post-quantum secure」は「既知の効率的量子攻撃がなく、公開鍵代数仮定を用いない」という意味で読むべきである。

1.4 2018 年時点の比較軸

原論文は、hPKC/pairing 系、離散対数系、対話証明系、MPC-in-the-head 系、IVC 系を、prover scalability、verifier scalability、transparency、post-quantum security の四軸で比較した。これは当時の研究状況を反映するが、2026 年の設計空間には、universal/updatable SRS、IPA polynomial commitment、folding schemes、hash-based PCS、binary-field SNARK などが加わる。したがって原論文図 2 は歴史資料として読み、第 24～29 章の比較で更新する。

	prover scalability (quasilinear time)	verifier scalability (polylogarithmic time)	Transparency (public randomness)	Post-quantum security
hPKC	Yes	Only repeated computation	No	No
DLP	Yes	No	Yes	No
IP	Yes	No	Yes	No
MPC	Yes	No	Yes	Yes
IVC+hPKC	Yes	Yes	No	No
ZK-STARK	Yes	Yes	Yes	Yes

原論文図2：2018年時点の汎用ZK方式の理論比較。緑が著者らの評価で有利な性質。

背景注

本論文の著者らは「verifierが入力されたプログラム記述を読むコスト」を statement size として分離する。uniform computation ではプログラムが短く、実行時間 T が巨大でも verifier は $|P| + \text{polylog } T$ でよい。一方、明示的な N 制約 R1CS を入力する場合、行列を読むだけで $\Omega(N)$ が必要である。この差が STARK と Aurora の比較で決定的になる。

2. 二進有限体・Reed-Solomon 符号・低次数拡張

2.1 二進有限体 $GF(2^n)$

論文は主に $F = GF(2^n)$ を使う。加法はビット列の XOR であり、乗法は既約多項式 $h(X)$ を法とする多項式積で実現される。標準基底 $1, g, \dots, g^{n-1}$ を選べば、 F は $GF(2)$ 上の n 次元ベクトル空間でもある。この二重性により、評価領域を加法部分空間またはその coset として取り、additive FFT を使える。

$$GF(2^n) \cong GF(2)[X] / (h(X)), \quad \deg h = n, \quad h \text{ irreducible}$$

実装ではフィールド表現、既約多項式、原始元、部分空間基底を両者が共有する。これは秘密 setup ではなく、計算パラメータから決まる公開記述である。

2.2 アフィン部分空間と消失多項式

$V \subset F$ を $GF(2)$ 線形部分空間、 $a \in F$ を shift とすると、 $H = a + V$ はアフィン部分空間である。 H 上で 0 になるモニック多項式 $Z_H(X) = \prod_{h \in H} (X - h)$ を消失多項式という。線形部分空間の消失多項式は linearized polynomial であり、次数は $|H|$ と大きくても非零係数が少なく、評価が効率的である。

$$Z_H(X) = \prod_{h \in H} (X - h), \quad Z_H(x) = 0 \Leftrightarrow x \in H$$

2.3 Reed-Solomon 符号

評価領域 $L \subset F$ と rate $\rho \in (0, 1)$ に対し、 $RS[F, L, \rho]$ は次数 $< \rho|L|$ の単変数多項式を L 上で評価した関数全体である。符号距離は少なくとも $1 - \rho$ であり、異なる二つの次数 $< \rho|L|$ 多項式は L 上の高々 $\rho|L| - 1$ 点でしか一致しない。

$$RS[F, L, \rho] = \{ (p(\alpha))_{\alpha \in L} : \deg p < \rho|L| \}$$

STARK で重要なのは「正しい符号語か」ではなく、「受け取った関数が符号から δ 以上離れているか」を少数 query で判定する proximity testing である。prover は全 oracle を構成するが、verifier は oracle 全体を読まない。

2.4 低次数拡張 LDE

実行トレースが H 上の値 $f: H \rightarrow F$ として与えられたとき、 H 上で f に一致する最小次数多項式 p を補間し、より大きな評価領域 L 上で p を評価する。これが low-degree extension である。生データを冗長な符号語へ変換するため、局所 query から全体の代数的一貫性を検査できる。

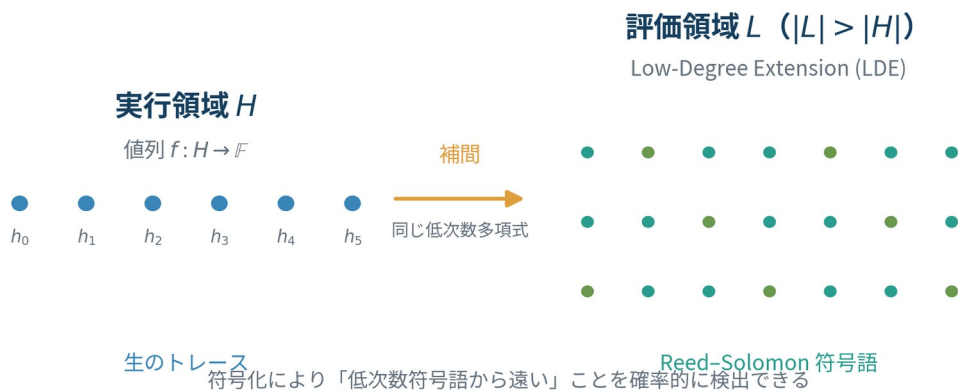


図 1-1：実行領域 H の値を同じ低次数多項式で大きな領域 L へ延長し、RS 符号語とする。

2.5 additive FFT

補間と多点評価を素朴に行うと二乗時間になる。論文は Lin–Chung–Han 型の additive FFT を使い、アフィン部分空間上の LDE を $O(N \log N)$ field operations で計算する。原論文は対象領域に対して約 $3|L|\log|L|$ の明示的演算数を示し、旧 additive FFT の $O(N \log N \log \log N)$ から改善したと述べる。

FFT は prover の主ボトルネックであり、漸近記法だけでは不十分である。メモリアクセス、coset 配置、SIMD、cache locality、out-of-core 戦略、複数列の batching が実時間を支配する。

3. IOP、STIK、STARK の形式

3.1 IOP の二相構造

Interactive Oracle Proof では、各ラウンドで prover が長い oracle message を送ったとみなし、verifier はその全体ではなく選んだ位置だけを読む。公開コイン IOP では verifier のメッセージは公開乱数であり、対話部分を Fiat–Shamir で非対話化しやすい。概念的には COMMIT 相と QUERY 相に分かれる。

- **COMMIT 相**：prover が oracle を固定し、verifier が後続のランダム challenge を生成する。
- **QUERY 相**：verifier が少数位置を選び、oracle 値と局所整合性を検査する。
- **実装時**：oracle は Merkle root でコミットされ、query answer には authentication path が付く。

3.2 STIK

STIK は Scalable Transparent IOP of Knowledge の略で、本論文が情報理論的レベルで構成する対象である。透明性、prover/verifier 双方のスケラビリティ、知識抽出、privacy preservation を要求する。IOP そのものの健全性は計算量無制限 prover に対して成立し得る。

性質	形式的意味
transparent	verifier のメッセージと query が public random coins
scalable verifier	$\text{poly}(n, \log T, \log 1/\epsilon)$
scalable prover	$T \cdot \text{poly}(n, \log T, \log 1/\epsilon)$
proof of knowledge	受理確率が knowledge error を超えれば witness を抽出
privacy preservation	transcript を完全分布一致で sample する simulator

3.3 STARK

STARK は STIK を暗号的に実装した Scalable Transparent ARGument of Knowledge である。oracle へ自由にアクセスできる理想モデルを、Merkle commitment とハッシュで実装すると、無制限 prover に対する proof ではなく、計算量制限 prover に対する argument になる。

- **interactive STARK**：衝突耐性ハッシュで oracle をコミットし、実際に対話する。
- **non-interactive STARK**：random oracle model で challenge を transcript から生成する。
- **ZK-STARK**：元の STIK が perfect ZK であれば、実装後は通常 computational ZK となる。

用語上の注意

「STARK proof」と呼ばれていても、実装された非対話プロトコルは暗号仮定に依存する argument である。論文は情報理論的 STIK と暗号的 STARK を分離している。この区別は、soundness proof、QROM、hash collision、knowledge extraction を議論する際に重要である。

4. AIR：計算を局所多項式制約にする

4.1 定義

Algebraic Intermediate Representation (AIR) は、長さ T 、幅 w の実行トレースと、その隣接行に作用する低次数制約からなる。状態 $s_t = (w_1(t), \dots, w_w(t))$ と次状態 s_{t+1} が正しい遷移であることを、有限体上の多項式 $P_1, \dots, P_s$ によって表す。

$$P_1(s_t, s_{t+1}) = \dots = P_s(s_t, s_{t+1}) = 0 \quad \text{for } t=0, \dots, T-2$$

さらに、初期値、公開入力、公開出力、固定定数を boundary constraints $B = \{(i, j, \alpha)\}$ として、 $w_j(i) = \alpha$ を要求する。原論文の形式定義では、各 P_i の zero/nonzero を入力とする monotone Boolean circuit C も許し、AND/OR で複合的な遷移条件を表す。実装例の多くでは単純な共通零点条件に近い。

AIR 実行トレース：行=時刻、列=代数レジスタ

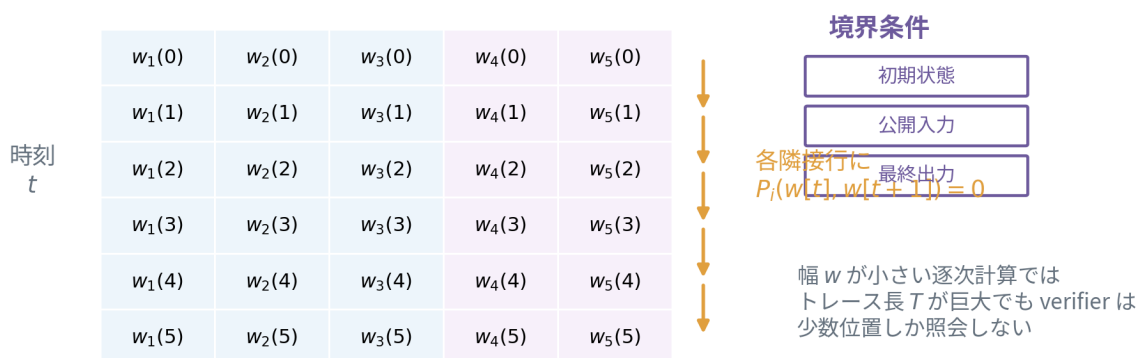


図 I-2：AIR 実行トレース。各行が時刻、各列がレジスタで、隣接行に局所多項式制約を課す。

4.2 複度パラメタ

記号	意味	効率への影響
T	cycle count／トレース長	prover 時間と oracle 長にほぼ線形
w	state width／レジスタ数	列数、query 時の開示量、space-bounded verifier
$s= P $	制約多項式数	ALI 以前の合成コスト
$d=\max \deg P_i$	AIR 次数	composition polynomial の次数と blowup
$ B $	境界制約数	verifier 前処理と公開入力処理
$\text{Tarith}(P)$	制約評価演算数	composition oracle 生成コスト

AIR の「良さ」は、算術回路の gate count だけでは測れない。幅が小さく、遷移次数が低く、1 cycle あたりの制約評価が安く、メモリアクセスを局所化できることが重要である。例えば同じハッシュでも、ビット演算を素体上で模倣するか、二進体上で native に表すかで AIR の幅・次数・cycle 数が大きく変わる。

4.3 AIR と R1CS の差

R1CS は明示的な制約行列 A, B, C と assignment z に対し $(Az) \cdot (Bz) = Cz$ を要求する non-uniform 表現である。AIR は短い遷移規則を長い時間軸に反復する uniform 表現である。したがって、長い同型ステップを繰り返す VM・ハッシュチェーン・データ走査は AIR に自然であり、複雑な一回限りの回路や既存 R1CS コンパイラ資産は R1CS に自然である。

設計者の視点

AIR 化の仕事は「計算を多項式に変える」だけではない。range constraint、bit decomposition、lookup、memory consistency、branching、padding、public output binding を、低次数かつ低幅に落とすコンパイラ設計である。STARK の性能差の多くは FRI より前の算術化で生じる。

5. APR : 実行・メモリ整合性を代数的グラフへ配置する

5.1 なぜ routing が必要か

空間が小さい逐次計算では、次状態は直前状態だけを見ればよい。しかし RAM 計算では、時刻 t の read が過去のどの write に対応するかは入力依存であり、単純な隣接行制約では検査できない。そこで、実行順の列とメモリアドレス順に並べ替えた列の双方を用意し、後者が前者の置換であることを証明する必要がある。

APR (Algebraic Placement and Routing) は、状態値をアフィン写像 $N(x) = ax + b$ で生成されるグラフ上に配置し、必要な「近傍」を低次数写像で参照可能にする。物理回路の placement and routing に似るが、最適化対象は配線長ではなく、polynomial degree、rate、overlap、evaluation domain である。

5.2 APR instance

APR instance は概略として $(F, \mathcal{I}, \mathcal{A}, \Phi, L, L_cmp, \bar{\rho}, \rho_cmp)$ からなる。 $\mathcal{I}$ は algebraic register index、 $\mathcal{A}$ は各 register に作用する affine neighbor maps、 Φ は局所制約、 L は witness 評価領域、 L_cmp は composition 評価領域、 ρ は各 RS code の rate である。witness は register ごとの関数 $w_\tau: L \rightarrow F$ で、各 w_τ が指定 rate の RS 符号語であり、各 $\phi \in \Phi$ を近傍値へ適用した composition も指定 rate の RS 符号語であることを要求する。

$$w_\tau \in RS[F, L, \rho_\tau], \quad \phi_N[w] \in RS[F, L_cmp, \rho_cmp] \quad \text{for all } \tau, \phi$$

5.3 space-bounded reduction

space-bounded 計算では、時刻を $GF(2)$ 部分空間 H 上に配置し、identity と二種類の cyclic neighbor map で現在・次状態を参照する。境界値は多項式 $Z_{\{B,j\}}$ と $E_{\{B,j\}}$ に吸収し、ランダム補間多項式 Q_j を $Z_{\{B,j\}}$ で割ったものを APR witness とする。評価領域 L を H から分離するため、分母は L 上で 0 にならない。

$$w_j^{APR}(x) = (Q_j(x) - E_{\{B,j\}}(x)) / Z_{\{B,j\}}(x)$$

この構成は、AIR 境界条件を自動的に満たしつつ、 Q_j に余分な自由係数を与えてゼロ知識化する。制約側では、遷移多項式が H の各点で 0 になるため、消失多項式 Z_H で割った商が低次数になるという factor theorem を利用する。

5.4 general computation と De Bruijn routing

一般計算では、実行順とメモリ順の間の置換 π を back-to-back De Bruijn network に route する。ネットワークの各 stage をアフィン写像として埋め込み、control register が経路を指定し、data register が同じ経路を通ることを低次数制約で保証する。これにより、任意置換を $O(T \log T)$ サイズの構造で表現できる。

代償は、witness 長が $O(T \log T)$ 、LDE 領域が $O(T \log T)$ 、prover が $O(T \log^2 T)$ へ増えることである。現代の STARK 実装では permutation argument、grand product、lookup、sorted memory table など別の表現も使われるが、「時間順とメモリ順の整合性を代数的に証明する」という本質は同じである。

APR の役割

APR は証明系の装飾ではなく、uniform computation を低次数局所制約へ落とすための配線層である。Aurora が R1CS 上で routing を避けられるのは、制約行列自体が明示入力であり、lincheck が任意線形写像を直接扱うか

らである。

6. ALI：多数の制約を二つの RS 近接問題へ縮する

6.1 旧方式の最大次数

旧来の arithmetization では、実行トレース全体を一つの多項式に詰め、各制約を個別に扱うため、最大次数は概ね次式になった。

$$d_{\text{max}}^{\text{old}}(n) = n \cdot c \cdot w \cdot d + n \cdot c \cdot s$$

第1項の w は全レジスタを一行に連結したこと、第2項の s は制約を個別に合成したことに由来する。本論文は、register ごとの符号化で w を除き、ランダム線形結合で s 項を除く。

$$d_{\text{max}}^{\text{STARK}}(n) = n \cdot c \cdot d$$

	$d_{\text{old}}^{\text{max}}(1)$	$d_{\text{ZK-STARK}}^{\text{max}}(1)$	$d_{\text{old}}^{\text{max}}/d_{\text{ZK-STARK}}^{\text{max}}$
SHA2	6323922	41382	18491
AES128+DM	39504	384	6584
Rij160+DM	49996	464	6896
DPM	78988	496	10192

原論文図5：旧方式と本方式の最大次数。表の比率は 6,584~18,491 倍。

原文の数値不整合

本文は改善率を「 $6.5 \times 10^4 \sim 1.8 \times 10^5$ 」と記すが、図5の数値と式(3)(4)から得られる比は約 $6.5 \times 10^3 \sim 1.85 \times 10^4$ である。表の計算が一桁小さく、本文の指数に誤植がある可能性が高い。本書では表と式の一致を優先する。

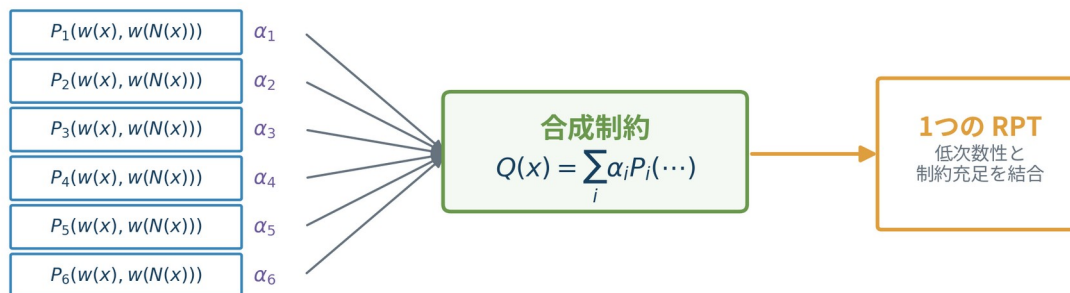
6.2 register-based encoding

幅 w の各レジスタ列を別々の RS 符号語として持つと、各多項式の次数は T 程度に下がる。しかし w 本すべてを個別に FRI 検査すれば通信量が増える。そこで verifier がランダム係数を選び、全 register codeword のランダム線形結合だけを近接性検査する。どれか一つが code から遠ければ、結合も高確率で code から遠い。

6.3 constraint linking

同様に、 Φ に含まれる多数の composition constraint をランダム線形結合し、一つの composition oracle $g^{\wedge}(0)$ へまとめる。ALI は最終的に、assignment 側 $f^{\wedge}(0):L \rightarrow F$ と composition 側 $g^{\wedge}(0):L_{\text{cmp}} \rightarrow F$ の二つを生成し、それぞれを RS proximity testing へ渡す。

ALI：多数の局所制約をランダム線形結合で「リンク」する



誤りが 0 でなければ、ランダム係数で打ち消される確率は概ね $1/|F|$ 。距離保証の詳細が健全性解析の核心。

図 I-3：ALI は多数の局所制約をランダム線形結合し、一つの合成制約へ圧縮する。

6.4 soundness の形

APR instance が unsatisfiable なら、任意の assignment oracle に対し、verifier のランダム係数を選んだとき、 $f^*(0)$ または $g^*(0)$ の少なくとも一方が対応 RS code から一定距離以上離れる。ただし原論文の定理 B.15 が無条件に保証する距離は $\delta/(2\zeta)$ と保守的であり、具体実装はより強い blockwise distance の予想を用いて query 数を減らす。

$$\Pr_R[\text{dist}(f^*(0), RS) < \delta/(2\zeta) \text{ and } \text{dist}(g^*(0), RS) < \delta/(2\zeta)] \leq 1/|F|$$

ζ は neighbor overlap と $|L|/|L_{\text{cmp}}|$ から決まり、space-bounded reduction では $\zeta=5$ 、general reduction では $\zeta=21$ となる。overlap が大きいほど、一つの assignment 誤りが複数 composition 位置に影響する構造の解析が難しくなる。

7. FRI：線形時間 prover と対数時間 verifier

7.1 低次数検査の課題

verifier は、oracle $f:L \rightarrow F$ が次数 $< \rho|L|$ の多項式評価に近いかを、 L 全体を読まずに判定したい。単純に多項式係数を送らせると証明が長く、PCP 系 low-degree test は prover が準線形以上で、具体定数も大きかった。FRI は、符号語を段階的に折り畳み、各段の整合性を少数 query で検査する。

7.2 folding の直観

評価領域を小さな coset に分け、各 coset 内の値からランダム係数 β を使って新しい短い oracle を作る。正しい低次数多項式なら、折り畳み後も次数上限がほぼ半分の多項式評価になる。遠い関数が毎段不正を隠し続ける確率は小さい。

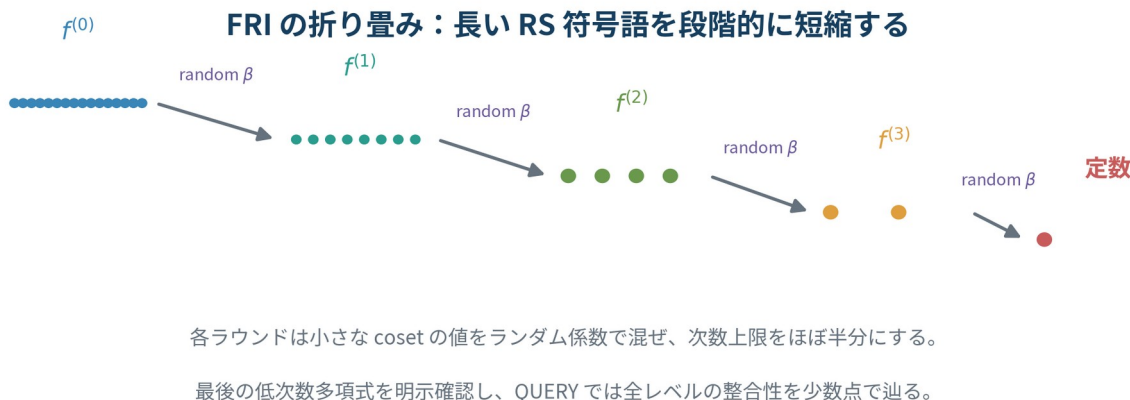


図 I-4：FRI folding。長い符号語をランダム challenge で段階的に短縮し、最後に低次数性を直接確認する。

7.3 原論文の FRI パラメータ

項目	原論文の主張
prover arithmetic	$< 6N$ field operations
proof-oracle length	$< N/3$ field elements
rounds	概ね $(\log N - R)/2$
query complexity	$2\ell \log N$
verifier arithmetic	$\leq 21\ell \log N$ field operations
soundness	$3N/ F + (1 - \min\{\delta, \delta_0\})^\ell$
parallel depth	各 message を PRAM で $O(1)$ 、全体 $O(\log N)$ cycles

N は block length、 R は rate $\rho=2^{-R}$ を表す整数、 ℓ は QUERY repetition である。2018 年の FRI 解析は後続の DEEP-FRI、FRI soundness 研究、STIR、WHIR により改善された。したがって上の定数は歴史的実装値として有用だが、現代プロトコルの最終選択には最新の tight soundness 解析を使うべきである。

7.4 COMMIT と QUERY の分離

FRI の soundness は、COMMIT phase の randomness で悪い関数が十分遠いまま残る確率と、QUERY phase で不整合位置を引く確率に分解できる。QUERY だけを繰り返せば prover oracle を増やさず error を下げられるため、証明サイズと prover 時間に有利である。この二相分解は Aurora でも重要である。

FRI を誤解しない

FRI は「多項式評価を証明する」完全な PCS ではなく、基本的には RS 符号への近接性を検査する IOPP である。評価主張、AIR 制約、sumcheck、composition polynomial などは別の代数プロトコルで FRI へ接続される。STARK 全体の性能は FRI 単体だけで決まらない。

8. 完全ゼロ知識と知識抽出

8.1 有界独立なランダム補間

秘密値を H 上で固定しながら、 H 外の評価値をランダムに見せるため、最小次数補間ではなく、余分な自由係数を持つランダム多項式を選ぶ。例えば $|H|$ 個の条件に対し次数 $< |H| + k$ の多項式を一様に選べば、 H 外の高々 k 個の評価は独立一様に分布する。評価領域 L を H と交差しない coset に取ることが重要である。

$$p(X) = p_0(X) + Z_H(X) \cdot r(X), \quad \deg r < k$$

p_0 は秘密値を補間する固定多項式、 r は一様ランダムである。 H 上では $Z_H=0$ なので p は秘密値を保ち、 L 上では $Z_H \neq 0$ なので r の乱数が評価値を隠す。

8.2 なぜ bounded independence だけでは足りないか

composition や商多項式を作ると、 Z_H 因子で割ることによりランダム部分が消え、秘密依存の低次数成分が露出することがある。したがって、個々の witness oracle をランダム化するだけでなく、検査対象の linear combination 自体へ独立な mask codeword を加える。これは Reed–Solomon proximity testing の random self-reducibility を利用する。

完全 ZK の構造：bounded independence と self-reducibility



各評価値をランダムに見せるだけでは除算・合成で乱数が消える。ランダム符号語を混ぜた主張へ自己還元し、検査対象自体をランダム化する。

図 I-5：完全 ZK。ランダム低次数拡張と mask codeword で、低次数検査が秘密トレースへ依存しない分布になる。

8.3 perfect ZK と computational ZK

IOP レベルでは、query bound 以下の悪意 verifier に対して transcript 分布を完全に再現する simulator を構成できる。Merkle commitment と random oracle で実装した後は、commitment hiding や random oracle programmability に依存するため、通常は computational ZK として表現される。

8.4 知識抽出

受理確率が knowledge error を超える prover から、まず FRI oracle を近い RS codeword へ復号し、ALI extractor で APR witness を得て、APR extractor で AIR execution trace を復元し、最後に元計算の nondeterministic witness を取り出す。抽出器は prover へ oracle access を持ち、必要に応じて複数回実行する。

FRI decoding → ALI extractor → APR extractor → AIR trace → original witness

ここで PoK は「実際の実装が必ずデータを保存していた」という物理的主張ではなく、受理させる戦略から有効 witness を計算的に抽出できるという暗号学的性質である。データ availability や現実世界の provenance は別問題である。

9. Merkle 木・BCS 変換・通信量

9.1 oracle commitment

理想 IOP では verifier が oracle の任意位置を読む。実装では prover が oracle entries を Merkle tree の leaf に置き、root を先に送る。challenge 後、query された leaf 値と authentication path を開示する。衝突耐性があれば、同じ root に対し異なる値を都合よく開くことは困難である。

9.2 通信量の分解

$$CC = q_{\text{total}} \cdot \log|F| + AP_{\text{total}} \cdot \lambda_{\text{hash}}$$

第 1 項は field element の query answer、第 2 項は authentication path の hash digest である。実測では後者が支配的になる。oracle の数学的 query complexity だけを最適化しても、Merkle tree の深さ・query 位置の重複・複数 oracle の配置が悪ければ証明は大きい。

9.3 行ハッシングと coset ハッシング

verifier が同一時刻の w 個の register をまとめて読むなら、その行を一つの leaf/subtree として配置し、一つの authentication path で開く。FRI が固定 subspace の coset をまとめて読むなら、その coset も一つの subtree に置く。これにより w 本または coset 内全要素ごとの認証路を避ける。Aurora の column hashing も同じ原理である。

9.4 Fiat-Shamir / BCS 変換

非対話化では、各 round の Merkle root と過去 transcript を hash し、verifier challenge を決定する。ドメイン分離、transcript serialization、field element encoding、rejection sampling、hash-to-field の bias、multi-proof path pruning を仕様化しなければ、抽象証明の安全性を実装へ移せない。

原論文実装は BLAKE2 ではなく、AES 命令を利用しやすい Davies-Meyer/AES128 を Merkle hash として用い、collision security を約 64 bit と見積もった。これは 2018 年 proof-of-concept の選択であり、現代の 128-bit 以上の運用では digest 長と量子余裕を再設計する必要がある。

第 II 部 原論文の注釈付き網羅訳

本文第 1～3 節と付録 A～G を、原文の順序・定理番号・図番号に沿って再構成する

10. 要旨と導入（原論文 pp.1-13）

10.1 要旨の再構成訳

人間の尊厳のため、医療・法科学データのような個人情報公開から守られなければならない。しかし秘密の覆いは、データを預かる制度が虚偽や不正を隠すためにも利用され得る。ゼロ知識証明は、個人の privacy を損なわずに institutional integrity を強制する暗号学的手段である。公共の信頼には、trusted party や trapdoor に依存しない透明性が必要であり、Big Data へ適用するには verification がデータサイズより速く拡張しなければならない。

本論文は、一般計算に対して透明性と指数的な verification speedup を同時に具体実装した最初の ZK-STARK を報告する。中心技術は IOP、線形時間の FRI、AIR/APR/ALI である。proof-of-concept では、警察が保持する秘密 DNA データベースに候補者プロフィールが含まれないことを、データベースも候補者 DNA も公開せず、外部 trusted party なしに証明する。proof はデータベースより短く、verification は全件走査より速い。

10.2 導入：公開データと秘密データ

公開データに対する計算完全性は、verifier が計算を再実行すれば確認できる。しかしこの素朴解は計算時間 T_C とデータサイズ $|D|$ に比例し、スケールしない。さらに D が秘密なら再実行自体ができない。従来の監査人は、秘密データを読んで代理検証するが、スケールしないだけでなく、賄賂・侵害・強制の single point of failure になる。

ハッシュコミットメント $\text{cmt}(D_t)$ は、ある時刻のデータ状態を短い fingerprint として公開台帳へ固定できる。求める proof system は、 cmt 、公開出力、公開 program に対して、秘密 D が commitment を開き、その D 上の計算結果が正しいことを証明する。verifier time と communication は、理想的には $\log T_C$ と $|\text{cmt}|$ 程度であり、少なくとも T_C と $|D|$ を下回る必要がある。

10.3 ZK-PCP から IOP へ

1990 年代の PCP theorem は、prover time を多項式に増やす代わりに、verifier time を polylogarithmic へ下げる可能性を示した。ZK-PCP は透明・汎用・秘匿・耐量子・知識性・scalable verification という望ましい性質を備えるが、古典的構成は proof 生成・保存が実用不能なほど巨大だった。

IOP は、PCP の oracle query と IP の複数 round を組み合わせる。相互作用により、prover は巨大な静的 PCP を一度に書く代わりに、challenge に応じて構造化 oracle を段階的に送れる。理論研究は prover time $T \cdot \text{polylog } T$ の ZK-IOP を示していたが、本論文はその具体効率を意味のある逐次計算で示す。

10.4 Main contribution (Section 1.1)

著者らは、透明で prover/verifier 双方が scalable な ZK-STARK を構成し、Merkle/ハッシュコンパイラで ZK-STARK として実装する。DPM において、(i) verifier time < naive time、(ii) communication < witness size を実測した。改善の核は FRI、新しい arithmetization、register ごとの RS 符号化、ALI である。

DPM の公開入力データベース commitment cmt 、候補者 profile commitment cm_p 、結果 α であり、秘密入力はデータベース D' と profile p' である。計算 C は、 $\text{commit}(D')=\text{cmt}$ 、 $\text{commit}(p')=\text{cm}_p$ 、 $\text{search}(D',p')=\alpha$ の三条件を検査する。 α は no match、partial match、full match のいずれかである。

10.5 縮率と値

$$\rho_{\text{time}}(n)=T_V(n)/T_C(n), \quad \rho_{\text{size}}(n)=CC(n)/|D(n)|$$

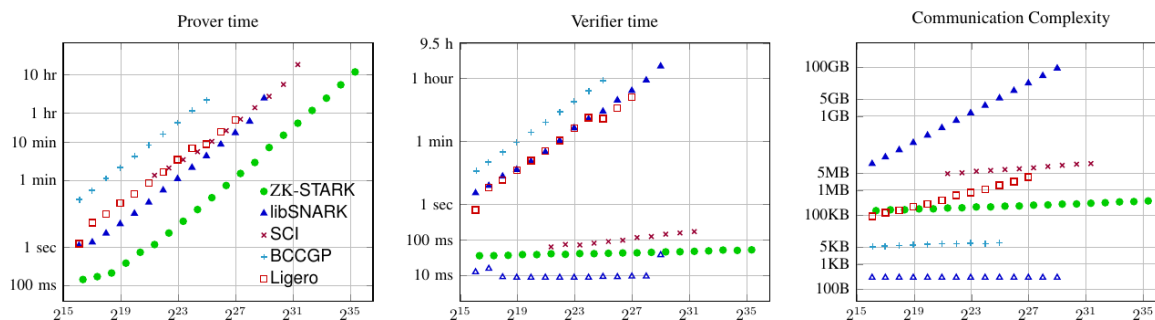
$\rho < 1$ なら proof system による圧縮が実現している。compression threshold θ は、以後すべての n で $\rho < 1$ となる最小 n である。DPM では $\theta_{\text{time}} \approx 2.8 \times 10^5$ 、 $\theta_{\text{size}} \approx 9 \times 10^3$ 。最大生成例 $n = 2^{20} \approx 1.05 \times 10^6$ （原文は約 1.14×10^6 と記すが 2^{20} は 1,048,576）で、40n bytes ≈ 40 MiB、論文記載では約 43 MB、time ratio 1/6、size ratio 1/100 である。

10.6 分散台帳への応用 (Section 1.2)

Scalability の側面では、単一または少数の prover node が台帳状態遷移の証明を準線形時間で作り、他 node が全計算・全状態を再実行せず検証できる。Privacy の側面では、当時 Zcash が用いた非透明 pairing-based SNARK を透明 STARK へ置き換える可能性を論じる。ただし原論文当時、STARK proof は SNARK より約 1000 倍長いとされ、aggregation・recursive proof・incrementally verifiable computation が課題だった。

10.7 2018 年の他方式比較 (Section 1.3)

hPKC/pairing 系は短い proof と高速 verification を持つが、circuit-specific setup が計算規模に比例し、秘密 trapdoor と量子脆弱性がある。DLP 系 (Bulletproofs 等) は透明だが量子脆弱で、一般回路 verification が線形。IP 系は透明で prover が scalable だが、逐次計算で verifier が非 scalable。MPC-in-the-head 系は透明・耐量子・prover 準線形だが、当時の Ligero では communication が $\sqrt{T}$ 程度。IVC+hPKC は一回の計算でも scalable verifier を得るが非透明・量子脆弱だった。



原論文図 3：同一 server・同一 subset-sum 構造での prover time、verifier time、communication 比較。

図 3 の「apples-to-apples」比較では、ZK-STARK prover が測定範囲で他方式より少なくとも約 10 倍速いと報告される。しかし security level、実装成熟度、言語、並列化、前処理の扱いが異なるため、普遍的順位ではなく、深く狭い回路に対する当該実装の比較として読むべきである。

10.8 回路換算と DPM パラメータ

他方式との比較のため、DPM を算術回路へ展開すると、depth=62n、width=81、witness=40n bytes、multiplication gates=1,467·62n=90,954n $\approx 2^{16.4}n$ となる。STARK のネイティブな複雑度は AIR width/cycles/degree で決まるため、gate count は比較用の派生指標である。

	State size w	Cycles c	Degree d	System size s	#× gates	#+ gates	total #× gates
SHA2	56	3762	11	1065	720	3194	2708640 $\approx 2^{21.3}$
AES128+DM	62	48	8	327	486	1033	23328 $\approx 2^{14.5}$
Rij160+DM	68	58	8	318	891	1390	51678 $\approx 2^{15.7}$
DPM	81	62	8	626	1467	1520	90954 $\approx 2^{16.4}$

原論文図 4：SHA2、AES128+DM、Rijndael160+DM、DPM の AIR 幅・cycle 数・次数・制約数。

注釈

原論文の性能主張は 2017～2018 年の実装・hardware・security parameter に基づく。現代方式との比較では、field、hash、proof compression、GPU、SIMD、recursion、batching を揃える必要がある。歴史的な「最速」を現在の絶対評価へ延長してはならない。

11. Methods (原論文 pp.14-18)

11.1 全体像

計算完全性主張「program C を公開入力 x で T step 実行した結果が a 」を、二つの RS proximity testing 問題へ還元し、FRI で解く。処理は、AIR 構築、APR reduction、ALI、FRI、Merkle 実装の順である。

11.2 FRI (Section 2.1)

$RS[F, S, p]$ に対し、oracle f が codeword か、すべての codeword から相対 Hamming 距離 0.1 以上離れているかを、少数 query で区別する。原論文は FRI を、prover $<6|S|$ field operations、verifier $<21\log|S|$ operations、parallel depth $\log|S|$ 、短い argument に適した構造を持つ最初の strictly linear RPT solution と位置づける。

11.3 AIR (Section 2.2)

AIR の polynomial set $P=\{P_1, \dots, P_s\}$ は current state $\vec{X}$ と next state $\vec{Y}$ の $2w$ 変数上にあり、 $P_i(\vec{x}, \vec{y})=0$ が valid transition を表す。主要パラメータは degree d 、width w 、system size s 、cycle count $c \cdot n$ である。hash function の AIR コストが DPM 全体を支配するため、 $GF(2^{64})$ に適した Rijndael/Davies-Meyer を選ぶ。

11.4 ALI と最大次数 (Section 2.3)

旧方式 $d_{\max}=n c w d+n c s$ に対し、本方式 $d_{\max}=n c d$ 。制約数 s を消すのが ALI、width w を消すのが register-based encoding である。複数 register をランダム線形結合して一回の RPT へ、複数 constraints もランダム線形結合して一回の composition RPT へまとめる。

11.5 LDE (Section 2.4)

LDE は H 上の trace value と同じ補間多項式を、大きい H' 上で評価したもの。binary field 上の additive FFT を使用し、prover arithmetic を strict $O(N \log N)$ へ抑える。degree 削減は domain サイズ・FFT 回数・Merkle tree サイズのすべてを縮小する。

11.6 APC/CC (Section 2.5)

communication は query value と authentication path の和。trace の同一行 w 要素を一 subtree へ置き、FRI query の affine coset も一 subtree へ置く。これにより one path per row/one path per coset を実現する。さらに AES instruction を使える Davies-Meyer hash を採用する。

$$CC = q_{\text{total}} \log|F| + AP_{\text{total}} \lambda$$

11.7 本文以降の構成 (Section 2.6)

Section 3 で STIK/STARK を定義し主定理を述べ、Appendix B で $AIR \rightarrow APR \rightarrow ALI \rightarrow FRI$ を統合する。Appendix C が APR、D が ALI、E が DPM AIR、F が SHA2 AIR、G が affine rearrangeable network を詳述する。

12. STIK/STARK の定義と主定理 (原論文 pp.19-23)

12.1 Computational Integrity relation

instance は (M, x, y, T, S) : nondeterministic Turing machine M 、input x 、output y 、time bound T 、space bound S 。witness は M が x から y へ到達する nondeterministic choices。LCI は NEXP-complete な universal computational integrity language である。space bound $S \leq T$ だが、小空間計算はより効率的なので別定理に分ける。

12.2 IOP 定義

r rounds で prover が oracle message を送り、verifier が public randomness を返す。proof length ℓ は全 oracle 長、query complexity q は読んだ entries 数、 t_P は oracle 生成時間、 t_V は verifier 計算時間。完全性 1、偽 statement に対する soundness error ϵ を要求する。

12.3 STIK 定義

transparent、fully scalable、proof of knowledge、privacy preserving の四条件。 $T(n)=\text{poly}(n)$ なら privacy preservation は perfect ZK となるが、 T が superpolynomial では simulator が $\text{poly}(T)$ であるため、 standard polynomial-time ZK ではなく witness indistinguishability と表現する。

12.4 Theorem 3.4 : space-bounded

$L \in \text{NTIME}(T, S)$ に対し、 perfect completeness、 $\text{soundness} \leq 2^{-\lambda}$ 、 knowledge error $O(2^{-\lambda})$ 、 $\text{rounds} \approx (\log T)/2 + O(1)$ 、 $\text{queries} = 36(\lambda + 2)(\log T + S + O(1))$ 、 verifier arithmetic $\approx O(\lambda(S + \log T))$ 、 prover $O(ST \log T)$ 、 proof $O(TS)$ field elements。 $S = \text{poly} \log T$ なら fully scalable。

12.5 Theorem 3.5 : 一般計算

$L \in \text{NTIME}(T)$ に対し、 $\text{rounds} \approx (\log T)/2$ 、 $\text{queries} O(\lambda \log T)$ 、 verifier $O(\lambda \log T)$ 、 prover $O(T \log^2 T)$ 、 proof $O(T \log T)$ 。
 $T = \text{poly}(n)$ なら perfect ZK。 原文の定理冒頭には $\text{err}(n) = 2^{-\{\lambda(n)\}}$ と読める植字があるが、直後の $\text{soundness} \leq 2^{-\lambda}$ および証明から負指数が正しい。

12.6 STIK から STARK へ

collision-resistant hash で interactive STARK、 random oracle model で non-interactive STARK へ変換する。
 blockchain を timestamp と randomness beacon として使い interactive mode を実装する可能性も論じる。選択は communication、 assumption、 use case による。

12.7 先行構成

PCP は 1-round IOP なので透明。 scalable verifier PCP、 quasi-linear PCP、 ZK-STIK for NP/NEXP という理論系列が先行し、2016 年の SCI が非 ZK STARK 実装、本論文が最初の ZK-STARK 実装と位置づけられる。

13. 実装測定の詳細読 (Appendix A, pp.31-32)

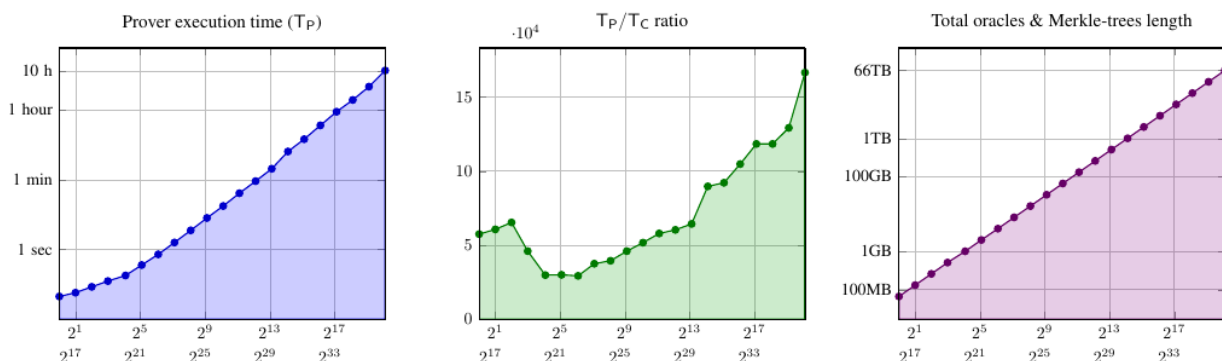
Appendix A は DPM (DNA Profile Match) に対する proof-of-concept 実装を、prover と verifier に分けて測定する。この付録は、漸近式だけでは見えない「メモリ階層」「Merkle 認証」「並列化」「非適応 query」の影響を読み取るうえで重要である。

測定値を読む際の前提

セキュリティ水準は 60 bit、有限体は $F_{2^{64}}$ 、Merkle hash は AES128 を用いた Davies–Meyer である。したがって、今日一般的な 128-bit security を想定した実装との数値比較には、そのまま用いない。また、後述する巨大 instance の verifier 値は、実 proof を生成して得た end-to-end 測定ではなく、非適応 verifier を利用した synthetic proof 上の測定を含む。

13.1 Prover 測定：時間overhead作業領域

原論文 Figure 6 は、(i) proof 生成時間、(ii) native execution に対する prover overhead、(iii) 全 IOP oracle と Merkle tree の理論上の総サイズを、database entries n の関数として示す。



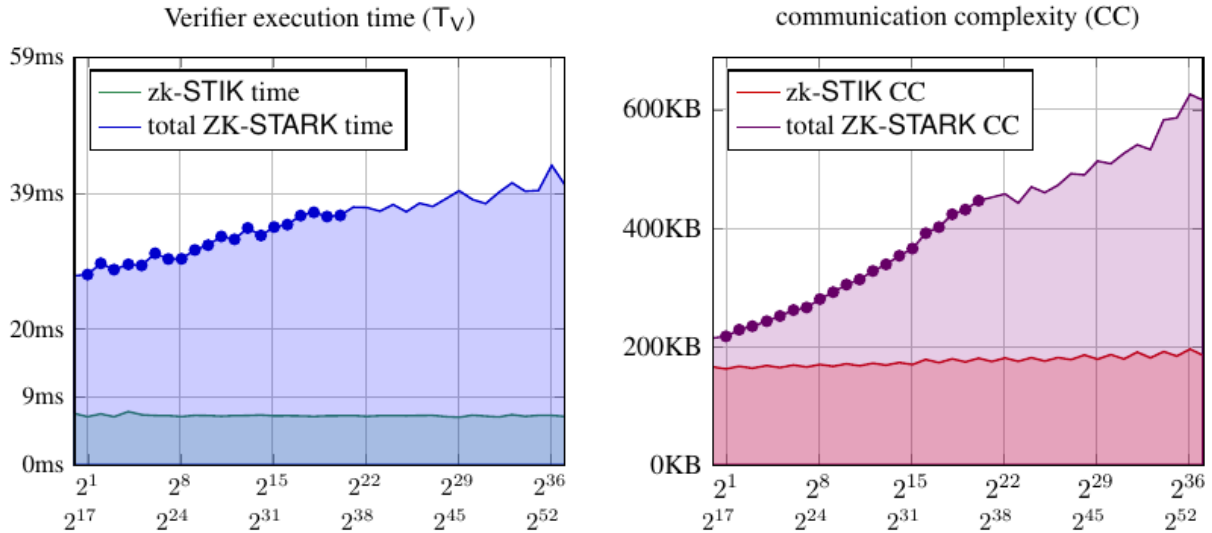
原論文 Figure 6 : DPM prover の実行時間、native 計算に対する倍率、全 oracle+Merkle tree サイズ。 $n \approx 2^{14}$ 付近で 768 GB RAM の閾値を越え、再計算・out-of-core 処理による phase transition が現れる。

左図の傾きは quasi-linear な prover 設計と整合する。一方、中央図の倍率は単調ではない。小 instance では thread 起動、同期、FFT plan、Merkle 初期化など固定費が支配し、中規模で相対 overhead が低下する。 $n \approx 2^{14}$ を越えると、右図の理論作業領域が物理 RAM 768 GB を越え、oracle の一部を保存せず必要時に再構成するため時間が跳ね上がる。

この挙動は、STARK prover の現実的性能が「field operations 数」だけでなく、memory bandwidth、NUMA、streaming layout、external-memory scheduling に強く依存することを示す。FRI そのものが線形でも、LDE と Merkle 化が大量の連続配列を扱うため、メモリ系が計算系より先に限界へ達しうる。

観測	数学的／実装的原因	設計上の含意
小規模で高い倍率	固定費、multi-threading overhead、cache warm-up	microbenchmark だけで prover 漸近性を評価しない
$n \approx 2^{14}$ で折れ曲がり	作業領域が RAM 閾値を超え、再計算と swap が発生	prover は time-space trade-off を明示的に設計すべき
oracle 総量が急増	LDE、mask、composition、各 FRI layer、Merkle nodes	streaming Merkle、layer-by-layer 消去、分散 proving が有効
並列化で約 2 倍改善	FFT・hash・pointwise 演算が並列化可能	core 数より memory bandwidth が飽和点を決めることが多い

13.2 Verifier 測定：IOP 本体より認証が支配



原論文 Figure 7: verifier 時間と communication complexity. 下側が idealized ZK-STIK (IOP) 部分、上側が Merkle 認証を含む ZK-STARK 全体。instance が大きくなるほど Merkle path の相対負担が増える。

verifier は二層に分解できる。第一は代数的検査を行う ZK-STIK verifier、第二は oracle 回答が事前 commit された Merkle root と整合することを確認する認証 verifier である。Figure 7 では、前者の時間・通信は小さく、後者が総コストの大部分を占める。oracle 長が増えると Merkle tree が深くなるため、認証 path 長は対数的に増える。

論文の verifier は non-adaptive で、query 位置は transcript と randomness から先に決まる。そのため、prover が生成できないほど巨大な n に対しても、ランダムに作った回答列を使って verifier の局所計算だけを測定できる。これは verifier scaling の microbenchmark として有用だが、proof 生成可能性、I/O、network 転送、完全な soundness を伴う end-to-end 実証ではない。

巨大 instance の 50 ms という数字の正しい解釈

$n=2^{36}$ 、witness 約 2.5 TB、約 2^{52} multiplication gates 相当でも verifier が 50 ms 未満という記述は、非適応 verifier を synthetic proof 上で走らせた測定である。実 prover は当時の hardware ではその proof を生成していない。したがって「巨大計算を 50 ms で完全に検証した」という意味ではなく、「query 生成と局所検査の時間が logarithmic に留まる」ことの実験的確認である。

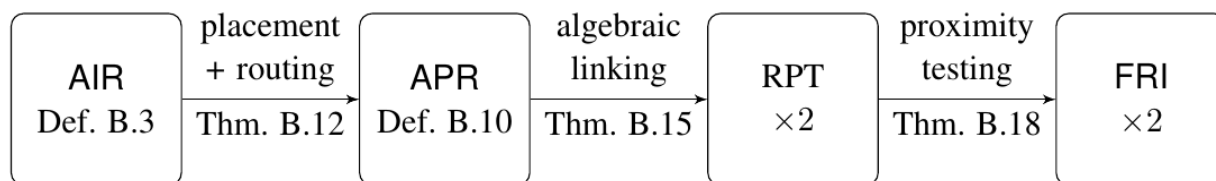
13.3 Figure 1 の compression threshold を再解する

本文で定義された time rate $\rho_{\text{time}}=T_V/T_C$ と size rate $\rho_{\text{size}}=CC/|D|$ は、proof system の実用域を「1 未満になったか」で表す。DPM では $\theta_{\text{time}} \approx 2.8 \times 10^5$ entries、 $\theta_{\text{size}} \approx 9 \times 10^3$ entries。 $n=2^{20}$ では $\rho_{\text{time}} \approx 1/6$ 、 $\rho_{\text{size}} \approx 1/100$ と報告される。

ただし、これらは特定 hardware、60-bit security、40-byte profile、特定 AIR、AES-DM Merkle という条件付きの閾値である。同じ protocol でも hash、field、security、query repetition、proof serialization、CPU/GPU、network が変われば閾値も変わる。「STARK は常に naive verification より速い」ではなく、「十分に大きい uniform sequential computation では漸近利得が具体化する」というのが正確な結論である。

14. AIR から ZK-STARK まで (Appendix B, pp.33–51)

Appendix B は論文全体の技術的背骨であり、自然な execution trace 表現を、最終的に二つの Reed–Solomon proximity testing problem へ落とす。本文の短い Methods を、定義・定理・複雑度・zero knowledge・knowledge extraction まで形式化する。



原論文 Figure 8 : AIR $\rightarrow$ APR $\rightarrow$ 2 個の RPT $\rightarrow$ 2 個の FRI。最初の矢印は deterministic algebraic placement/routing、中央は 1-round ALI、最後は FRI proximity test。

14.1 記法と有限体

全体を characteristic 2 の有限体 $F = \mathbb{F}_{2^n}$ 上で構成する。affine space は F -linear subspace V の coset $\alpha + V$ 。関数 $f: S \rightarrow F$ は evaluation vector として扱い、その interpolant を一意な次数 $< |S|$ の多項式と同一視する。LDE は小 domain 上の値を同じ interpolant で大 domain へ評価し直す操作である。

subspace S の vanishing polynomial $Z_S(X) = \prod_{s \in S} (X - s)$ は linearized polynomial であり、degree は $|S|$ でも非零係数は $\dim(S) + 1$ 程度に抑えられる。この疎構造が、verifier による Z_S 評価と affine-domain FFT を効率化する。

$$\text{LDE}_{\{S \rightarrow S'\}}(f) := \hat{f}|_{S'}, \quad \text{where } \deg(\hat{f}) < |S| \text{ and } \hat{f}|_S = f$$

Theorem B.2 は Lin–Chung–Han 型 additive FFT に基づき、複数 coset への LDE を概ね $3(n+1)|S|\log|S|$ field operations で計算できるとする。論文の strictly quasi-linear prover bound は、この FFT bound と FRI の linear prover を組み合わせて得られる。

14.2 BAIR : space-bounded computation の形式化

AIR instance $x = (F, T, w, P, C, B)$ は、field、time bound、state width、transition polynomials、制約の Boolean 結合 C 、boundary constraints B からなる。witness は w 個の register function $w_j: [T] \rightarrow F$ で、各時刻 t の state $w[t]$ が boundary と transition relation を満たす。

BAIR は binary field、 $T = 2^t - 1$ 、constraint degree $\leq 2^d$ という“binary-friendly”条件を課す。この power-of-two 近傍への padding は additive FFT と cyclic affine mapping を使うための構造化であり、元の計算意味論を変えるものではない。

BAIR 記号	意味	複雑度への寄与
$T = 2^t - 1$	execution length	domain dimension t 、round 数 $\approx t/2$
w	state width/register 数	space-bounded prover $O(wT \log T)$ 、verifier query に線形
$P = \{P_i\}$	transition polynomial 群	constraint evaluation cost、ALI randomness
d	degree bound 2^d	composition domain blow-up、RS rate
B	input/output 等の boundary	verifier 側補間・境界補正
k	ZK masking 余剰次数	bounded independence と domain size の trade-off
R	RS inverse-rate exponent	distance、FRI soundness、proof size

14.3 PAIR/BPAIR：一般 RAM 計算への拡張

一般計算では state width が T に近づき、単純 AIR witness が $\Omega(T^2)$ になりうる。そこで PAIR は、time-order transition constraints P_T と、memory access を address 順に並べ替えた permutation constraints P_π を併用する。witness は trace と permutation π 。これにより局所 transition と global memory consistency を分離する。

Lemma B.9 は、任意の $L \in \text{NTIME}(T)$ を BPAIR へ落とし、width $O(\log T)$ 、constraint 数・評価 cost $O(\log T)$ 、witness 生成 $O(T \log T)$ を得る。追加の $\log T$ は memory access を routing network で整列させる代償であり、Theorem 3.5 の prover $O(T \log^2 T)$ へつながる。

14.4 APR instance の三つの構造型

APR では各 register を別々の RS codeword w_τ として表し、affine neighbor map $N(x) = ax + b$ を通じて同時刻・次時刻・routing 先の値を局所的に参照する。constraint map ϕ は、ある x で必要な近傍値を受け取り 0 となるべき関係を記述する。

後段の soundness と ZK を支えるのが independence、distance、overlap の三量である。

性質	直観	使われる場所
κ -independence	任意の $\kappa L $ 個の座標が一様に見える masking 余地	perfect ZK simulator
δ -distance	不正 trace が RS code/constraint code から一定割合離れる	ALI→FRI soundness
Θ -overlap	composition point が参照する witness points の重なり上限	距離損失 $\zeta = \max\{4, 1 + \Theta(L / L_{\text{cmp}})\}$

14.5 ALI で二つの RPT へ

APR witness の全 assignment codeword と全 constraint image を個別に低次数検査する代わりに、verifier は random coefficients を送り、assignment 側を一つの $f^*(0)$ 、composition 側を一つの $g^*(0)$ に圧縮する。正しい instance なら双方が所定 RS code に属する。不正 instance なら高確率で少なくとも一方が一定距離離れる。

$$f^*(0)(x) = f_{\text{mask}}(x) + \sum_{\tau \in \text{mathcal{T}}} a_{\tau, w_\tau}(x),$$

$$g^*(0)(x) = g_{\text{mask}}(x) + \sum_{\phi \in \Phi} b_{\phi, \phi_{\text{mathcal{N}}}[\mathbf{w}]}(x)$$

Theorem B.15 の proved bound では、unsatisfiable APR に対し、確率 $1 - 1/|F|$ 以上で $f^*(0)$ または $g^*(0)$ が対応 RS code から少なくとも $\delta/(2\zeta)$ 離れる。knowledge extractor は両者が十分近い場合に decode して APR witness を回復する。 κ -independence があると、各 w_τ への query 数が $\kappa|L|$ 以下なら perfect ZK となる。

14.6 FRI を二回適用し、主定理を合成

$f^*(0)$ と $g^*(0)$ は domain と rate が異なるため、FRI を二回実行する。FRI は各 round で degree bound と domain size を同時に縮約し、最終的に小さな polynomial を直接確認する。Theorem B.18 の prototype パラメータでは、prover $< 6N$ field operations、proof oracle 長 $< N/3$ 、verifier $\leq 21\ell \log N$ field operations、queries $2\ell \log N$ 。

Theorem 3.4 は BAIR reduction (width $\approx$ space S) を使い、Theorem 3.5 は BPAIR reduction (width $\approx \log T$) を用いる。前者は small-space computation に優れ、後者は任意の time- T computation に fully scalable な bound を与える。

段階	space-bounded	general computation
元表現	BAIR, width $\approx S$	BPAIR, width $O(\log T)$
routing	cyclic successor のみ	time order + memory permutation
witness/oracle 規模	$O(TS)$	$O(T \log T)$
主要 LDE cost	$O(ST \log T)$	$O(T \log^2 T)$
verifier	$O(\lambda(S + \log T))$	$O(\lambda \log T)$
適する対象	deep/narrow、streaming、固定小	RAM/VM、large mutable memory

段階	space-bounded	general computation
	memory	

14.7 Appendix B.8 の具体 parameter

DPM 実装は Lemma B.7 の conjectured improved soundness を用いる。R=3、 $\lambda=60$ 、 $F=F_{2^{64}}$ 、AIR degree 8 ($d=3$)、ZK parameter $k=1$ 。FRI COMMIT repetition $s=2$ 、QUERY repetition は assignment 側 9、composition 側 22。dataset が小さいと $k=1$ では ZK query bound を満たさないため、論文は $n \geq 2^3$ を主対象とし、小 instance なら $k=3$ で足りるとする。

証明済み定理と prototype parameter の区別

Theorem 3.4/3.5 の存在・漸近 bound は proved analysis で成立する。一方、実装の query 数を減らした Lemma B.7 は Conjectures B.17 (ALI) および B.19 (FRI) に依存する。したがって、実験値を「全 parameter が査読済みの厳密 soundness bound」とみなしてはならない。現代の STARK 実装でも、security estimator がどの soundness theorem に依存するかを追う必要がある。

15. Algebraic Placement and Routing の詳細 (Appendix C, pp.52-62)

Appendix C は、計算 trace の時間順序・memory 順序を、有限体上の affine graph へ埋め込む具体構成を与える。本節では式の意味を保ちながら、space-bounded 版と general 版を分けて解説する。

15.1 Space-bounded 版：cyclic affine map で時間を表す

$T=2^t-1$ とし、degree- t primitive polynomial ζ を用いて、時刻 i を $g^i \bmod \zeta(g)$ で field element へ対応させる。 $H = \text{span}_{\mathbb{F}_2}\{1, g, \dots, g^{t-1}\}$ を time domain とし、 H を二つの affine half H_0, H_1 へ分ける。successor は half ごとに異なる affine map で表せる。

$$n_{\text{id}}(x) = x, \quad n^{\text{cyc}}_b(x) = g \cdot x + b \cdot \zeta \quad (b \in \{0, 1\})$$

H_0 上では通常の gx が次時刻を表し、wrap-around が必要な H_1 上では $gx - \zeta$ (characteristic 2 では $+\zeta$) が modular reduction 後の次時刻を表す。したがって「現在値」と「次値」を、degree-1 map を通じて同じ RS polynomial から取得できる。

15.2 Boundary condition を quotient へ埋め込む

register j の boundary constraints (i, j, α) を満たすため、 $Z_{\{B, j\}}$ を指定時刻で消える vanishing polynomial、 $E_{\{B, j\}}$ をその時刻で値 α を取る interpolant とする。APR witness w_j^{APR} から実 register polynomial Q_j を次式で再構成する。

$$Q_j(X) = w_j^{\text{APR}}(X) \cdot Z_{\{B, j\}}(X) + E_{\{B, j\}}(X)$$

honest prover は、trace 値に一致する randomized low-degree polynomial Q_j を選び、 $w_j^{\text{APR}} = (Q_j - E_{\{B, j\}}) / Z_{\{B, j\}}$ を L 上で送る。 L は boundary roots を含む H と disjoint なので division は well-defined。この形は、boundary を別途全点検査する代わりに、polynomial identity へ吸収する典型的な quotient technique である。

15.3 Transition quotient

各 AIR polynomial $P \in \mathcal{P}$ に対し、 H_0 と H_1 で別々の transition quotient を作る。例えば H_0 側は概念的に

$$\Phi_{\{P, 0\}}(X) = \frac{X(X-1)}{Z_{\{H_0\}}(X)} \cdot \text{P}\bigl(Q_1(X), \dots, Q_w(X), Q_1(gX), \dots, Q_w(gX)\bigr)$$

となる。 $X(X-1)$ は、 H_0 に含まれるが transition を要求しない端点 $0, 1$ を除外する補正である。honest trace では分子が H_0 の該当全点で 0 となるため $Z_{\{H_0\}}$ で割り切れ、quotient は低次数多項式になる。逆に不正 transition が一つでもあれば、低 rate RS codeword として全 domain で成立する identity と矛盾する。

局所制約から global polynomial identity へ

AIR では「各時刻 t で $P(\text{state}_t, \text{state}_{t+1}) = 0$ 」という T 個の条件がある。APR はこれを『一つの多項式が vanishing polynomial で割り切れる』という global algebraic condition へ変換する。STARK の核心は、全 T 点を読む代わりに、この低次数性を random query で検査する点にある。

15.4 完全性・soundness & knowledge extraction

完全性は、honest trace なら各 transition polynomial が H 上で消え、factor theorem により quotient が多項式となることから従う。soundness は逆向きで、APR witness が所定 rate の RS codeword で、すべての composition quotient も所定 rate なら、RS polynomial の一意性により H 上の boundary と transition が成立する。

extractor は APR codeword を interpolate し、 $Q_j = w_j^{\text{APR}} Z_{\{B, j\}} + E_{\{B, j\}}$ を H 上で評価して AIR trace を回復する。したがって knowledge property は、単なる accept/reject だけでなく、説得力のある prover から実際の execution trace を decode できることに由来する。

15.5 General 版 : memory consistency と permutation

RAM 計算では、隣接時刻だけを調べても memory の read が最後の write を参照したことを保証できない。PAIR は trace を time order と address order の二つで見る。time order では CPU transition、address order では同一 address の access sequence と value propagation を検査する。

unknown permutation π を verifier へ明示すると通信が線形に増えるため、prover は back-to-back De Bruijn network 上の routing を witness として与える。network は任意 permutation を実現でき、各 node は constant-degree の local switch constraint だけを持つ。この routing trace 自体を affine graph へ埋め込むことで、global permutation を local algebraic constraints へ還元する。

要素	役割	代数化
time trace	命令実行の逐次整合性	cyclic successor affine maps
sorted trace	address 順の read/write 整合性	permutation 後の隣接 constraint
control register	routing 先・switch 選択	binary/equality polynomials
back-to-back De Bruijn	任意 permutation の実現	11 程度の affine neighbor maps
copy constraint	data が selected edge へ正しく移動	q_cp 型 low-degree polynomial

Theorem B.13 では register 数は概ね $2(w+1)$ 、neighbor 数 $\leq 11|A|$ 。domain dimension に $\lceil \log(t+1) \rceil$ が加わり、routing cost $O(T \log T)$ と LDE cost $O(T \log^2 T)$ が一般計算の prover bound を決める。

一方、DPM のように small-space かつ streaming な計算では、memory permutation を持ち込む必要がない。本論文が DPM を benchmark に選んだ理由の一つは、space-bounded theorem の利点を最も明瞭に示せるためである。

15.6 Degree 設計と“register ごとに一 codeword”

旧構成は $w \times T$ の trace 全体を一つの $\text{degree} \approx wT$ polynomial に直列化していた。本論文は各 register を $\text{degree} \approx T$ の別 codeword とし、低次数検査は random linear combination 一つへ集約する。これにより最大 composition degree から w が消える。

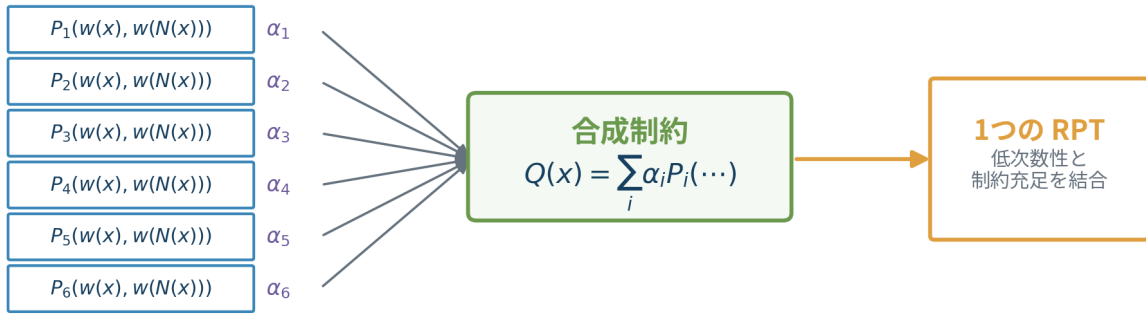
$$d_{\text{old}}^{\max} \approx T, w, d+T, s, \quad \text{vs.} \quad d_{\text{STARK}}^{\max} \approx T, d$$

Figure 5 の表値から得られる改善率は、AES128+DM で約 6.58×10^3 、Rij160+DM で約 6.90×10^3 、DPM で約 1.02×10^4 、SHA2 で約 1.85×10^4 である。本文中の「 6.5×10^4 – 1.8×10^5 」は式と表より一桁大きく、植字上の誤りと考えるのが自然である。

16. Algebraic Linking IOP の詳細 (Appendix D, pp.63–70)

ALI は、多数の assignment codeword と多数の constraint codeword を、二つの低次数性主張へ圧縮する一回の randomized linking である。FRI そのものより目立たないが、本論文の最大 degree 削減と proof size 削減を可能にした中心技術である。

ALI：多数の局所制約をランダム線形結合で「リンク」する



誤りが 0 でなければ、ランダム係数で打ち消される確率は概ね $1/|F|$ 。距離保証の詳細が健全性解析の核心。

図 16-1：ALI の概念。複数 register codeword を random linear combination で assignment oracle $f^{\wedge}(0)$ へ、複数 constraint image を composition oracle $g^{\wedge}(0)$ へ圧縮し、それぞれ一回の RS proximity test へ渡す。

16.1 Protocol schedule

1. Prover は、APR assignment codewords に加え、zero-knowledge 用 masking codeword 等を一つの interleaved oracle $O_{\text{assignment}}$ として送る。
 2. Verifier は $2|\mathcal{I}| + |\Phi|$ 個の uniform field elements を送る。これらが assignment mixing、constraint mixing、linking の係数となる。
 3. 両者は $O_{\text{assignment}}$ から virtual oracle $f^{\wedge}(0):L \rightarrow F$ と $g^{\wedge}(0):L_{\text{cmp}} \rightarrow F$ を局所的に計算する。
 4. $f^{\wedge}(0)$ が $RS[F, L, \rho_{\text{max}}]$ に、 $g^{\wedge}(0)$ が $RS[F, L_{\text{cmp}}, \rho_{\text{cmp}}]$ に近いことを、それぞれ FRI で検査する。
- 重要なのは、 $f^{\wedge}(0), g^{\wedge}(0)$ を完全に別配列として保存する必要はなく、query 点で元 oracle の少数座標から計算できる点である。ただし honest prover は FRI commit を作るため、実際には virtual oracle 全体を生成・Merkle 化することが多い。

16.2 Random linear combination の soundness

assignment 側でどれか一つの register が所定 RS code から離れている場合、random coefficients による線形結合も高確率で低次数 code から離れる。これは有限体上の Schwartz–Zippel 型議論、linear code の距離、interleaved code の random projection を組み合わせたものと見なせる。

constraint 側では、すべての register が低次数でも APR constraint を満たさない場合、少なくとも一つの $\phi_N[w]$ が constraint code から離れる。random linear combination により、その不整合が偶然相殺される確率を $1/|F|$ へ抑える。

$$\Pr_{R[\Delta(f^{\wedge}(0)), RS] < \Delta/(2\zeta)} \wedge \Delta(g^{\wedge}(0)), RS < \Delta/(2\zeta)} \leq |F|^{-1}$$

$\zeta = \max\{4, 1 + \Theta(|L|/|L_{\text{cmp}}|)\}$ は neighbor overlap による距離損失を表す。space-bounded APR では $\Theta=1$ 、 $|L|/|L_{\text{cmp}}|=4$ なので $\zeta=5$ 。general APR では $\Theta=5$ となり $\zeta=21$ 。この差が Theorem 3.5 で query constant を大きくする。

16.3 Perfect zero knowledge

APR reduction は、各 register polynomial を trace 値に条件付けた random high-degree polynomial から sampling する。自由係数が十分多ければ、 L 上の任意の $\kappa|L|$ 個の evaluation は uniform independent になる。ALI verifier が各 register へ行う distinct query 数をこの閾値未満に抑えると、simulator は witness を知らずに同一分布の回答を生成できる。

ALI の ZK theorem が任意の RS-IOPP subprotocol に対して成り立つ点は重要である。FRI 自体を zero knowledge 化する必要はなく、FRI へ入力される codeword を事前に random self-reduction で mask しておけばよい。これは Aurora にも継承される設計原理である。

16.4 Arithmetic complexity

ALI prover cost は概ね、各 assignment register の LDE cost $|\mathcal{A}||L|(3\log|L|+5)$ と、composition 評価 $2|L_{\text{cmp}}|(T_{\text{arith}}(\Phi)+|\Phi|+1)$ の和である。verifier の interactive phase は random field elements を sample するだけで、arithmetic cost はほぼ 0。実際の verifier cost は FRI query evaluation と Merkle 認証へ移る。

コスト源	漸近量	実装上のボトルネック
register LDE	$O(\mathcal{A} L \log L)$	FFT、memory bandwidth
constraint evaluation	$O(L_{\text{cmp}} \cdot T_{\text{arith}}(\Phi))$	pointwise field ops、cache
random mixing	$O((\mathcal{A} + \Phi) L)$	SIMD 化しやすい
FRI commit	$O(L)$	各 layer の hash と fold
Merkle	$O(\text{total oracle length})$	hash throughput、tree storage

16.5 Conjectured soundness の位置づけ

proved ALI theorem は Hamming distance を $\delta/(2\zeta)$ まで劣化させる。これをそのまま FRI へ渡すと query repetition が増える。著者らは、blockwise distance で見ると不正 word は実際には code distance 近く離れると予想し、Conjectures B.16/B.17 を置く。

さらに FRI についても、COMMIT 後の QUERY rejection probability が blockwise distance にほぼ一致するとする Conjecture B.19 を用いる。これらを仮定すると、field を $F_{2^{64}}$ に抑え、FRI query を 9/22 回にできる。

実務的 security review の要点

「hash collision bits」「Fiat-Shamir bits」だけでなく、(1) algebraic random combination の field-size error、(2) low-degree test の proved/conjectured error、(3) proximity gap、(4) query repetition、(5) Merkle binding、(6) quantum adversary に対する random-oracle loss を別々に積算する。一つの“security level”へ丸める前に、どの項が定理で、どの項が heuristic かを分離する。

17. DNA Profile Match の AIR (Appendix E, pp.71–79)

Appendix E は、DPM を単なる集合非包含ではなく、commitment verification、profile comparison、result aggregation を含む sequential program として AIR 化する。cryptographic hash の AIR cost が全体の大半を占める。

17.1 Rijndael S-box の代数化

AES/Rijndael の S-box は、 $GF(2^8)$ での multiplicative inverse (0 は 0 へ) に affine transformation を施す。Boolean circuit へ bit-blast するより、binary extension field 上で inverse relation $xy=1$ を直接 constraint 化すると degree と cycle を抑えられる。

$$\begin{aligned} & (INV1_{(t)}P00_{(t)} + 1)(P00_{(t)} \wedge INV1_{(t)}) \wedge \\ & (W11_{(t)} + INV1_{(t)}^4) \wedge (W12_{(t)} + W11_{(t)}^4) \wedge (W13_{(t)} + W12_{(t)}^4) \wedge \\ & (P00_{(t+1)} + c_0 \cdot INV1_{(t)} + c_1 \cdot INV1_{(t)}^2 + c_2 \cdot W11_{(t)} + c_3 \cdot W11_{(t)}^2 \\ & \quad + c_4 \cdot W12_{(t)} + c_5 \cdot W12_{(t)}^2 + c_6 \cdot W13_{(t)} + c_7 \cdot W13_{(t)}^2 + b) \end{aligned}$$

原論文 Figure 9 : Rijndael S-box の state machine。有限体逆元計算と affine 変換を、少数 register ・ low-degree transition へ分解する。

AIR では一つの complex operation を一 cycle で扱う必要はない。中間 register を増やし複数 cycle へ分割することで、各 transition polynomial の degree を 8 以下に保つ。ここでは state width、cycle count、degree、constraint count の多目的最適化が必要となる。

17.2 AES round と Davies–Meyer hash

AES round は SubBytes、ShiftRows、MixColumns、AddRoundKey からなる。ShiftRows は permutation、MixColumns は linear map、AddRoundKey は binary field addition なので、AIR cost の中心は SubBytes と key schedule である。

Davies–Meyer compression は block cipher $E_m(x)$ を用い $H(m,x)=E_m(x) \oplus x$ とする。Merkle commitment と database hash chain の双方に同型の AES-friendly primitive を使うことで、CPU の AES-NI と AIR 効率を両立する。

$$DM(m, x) = E_m(x) \oplus x$$

Constraint polynomial for compression

$$\begin{aligned} & B0_{(t+1)} + P00_{(t)} + x \cdot P10_{(t)} + x^2 \cdot P20_{(t)} + x^3 \cdot P30_{(t)} \\ & \quad + x^4 \cdot P01_{(t)} + x^5 \cdot P11_{(t)} + x^6 \cdot P21_{(t)} + x^7 \cdot P31_{(t)} \end{aligned}$$

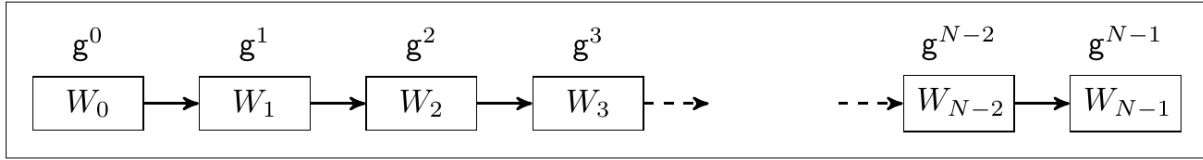
Constraint polynomial for Fermat's little theorem

$$(INV2_{(t)} + INV1_{(t)}^4) \wedge (INV3_{(t)} + INV2_{(t)}^4) \wedge (INV4_{(t)} + INV3_{(t)}^4) \wedge (INV1_{(t)} + INV4_{(t)}^4)$$

原論文 Figure 10 : Rijndael/Davies–Meyer compression function に対する AIR constraints の概略。

17.3 Database commitment の構造

各 DNA profile は 20 loci について二つの allele count を持つため、40 byte として表現される。database $D=(p_1, \dots, p_n)$ は逐次 hash chain または tree-like structure へ commit され、public commitment cmt と結び付けられる。候補者 profile p にも別 commitment cmp を置く。



原論文 Figure 11 : DNA database と commitment の data structure. 各 40-byte profile を hash 処理し、公開 commitment へ集約する。

証明 statement は、秘密 D', p' が public commitments cmt, cmp を開き、 D' に対する p' の search result が public α に一致すること。ZK により D', p' そのものは開示しない。

$$C(cmt, cmp, \alpha; D', p') = 1 \text{ iff } Com(D') = cmt \wedge Com(p') = cmp \wedge Match(D', p') = \alpha$$

17.4 Pair comparison と result semantics

profile comparison は 20 loci それぞれで、候補者の二 allele と database entry の二 allele を unordered pair として比較する。locus 単位の一一致数を集計し、full match、partial match、no match の三値へ写像する。

```

CheckPairs( $VAL_{1,1}, VAL_{1,2}, L_1, R_1, VAL_{2,1}, VAL_{2,2}, L_2, R_2, \dots, VAL_{10,1}, VAL_{10,2}, L_{10}, R_{10}$ )

I:  $t \leftarrow 2$ 
II: for  $j \in \{1, \dots, 10\}$  do
III:   if ( $VAL_{j,1} = L_j$ ) AND ( $VAL_{j,2} = R_j$ ) then Continue end if
IV:   if ( $VAL_{j,1} = R_j$ ) AND ( $VAL_{j,2} = L_j$ ) then Continue end if
V:    if ( $VAL_{j,1} = L_j$ ) OR ( $VAL_{j,1} = R_j$ ) OR ( $VAL_{j,2} = L_j$ ) OR ( $VAL_{j,2} = R_j$ ) then
VI:       $t \leftarrow 1$ 
VII:   else
VIII:     return 0
IX:   end if
X: end for
XI: return t

```

原論文 Figure 12 : allele pair の比較手続き。順序を無視した pair equality を低次数制約で判定する。

```

MatchingResult( $T_1, T_2, T$ )

I: if  $T_1 = T_2 = 2$  then return 2 end if
II: if ( $T_1 = 0$ ) OR ( $T_2 = 0$ ) then return  $T$  end if
III: if  $T = 0$  then return 1 end if

```

原論文 Figure 13 : 各 locus の比較結果から full/partial/no match を決定する手続き。

AIR では branch をそのまま扱わず、selector bit b に $b(b-1)=0$ を課し、 $b \cdot \text{expr}_1 + (1-b) \cdot \text{expr}_0$ の形で conditional update を代数化する。equality test には inverse witness を導入し、 $x=0$ か否かを低 degree constraints で表す。

17.5 Pair constraints と result constraints

$$\begin{aligned}
 & (\text{PAIR1}_{(t)} + \text{values}_{i,0}) \cdot (\text{PAIR1}_{(t)} + \text{values}_{i,1}) \cdot [\\
 & \quad (\text{PAIR2}_{(t)} + \text{values}_{i,0}) \cdot (\text{PAIR2}_{(t)} + \text{values}_{i,1}) \cdot \text{NewFlag}_{(t)} \wedge \\
 & \quad \text{LastFlag}_{(t)} \cdot (\text{NewFlag}_{(t)} + 1) \cdot [\\
 & \quad \quad ((\text{PAIR2}_{(t)} + \text{values}_{i,0}) \cdot \text{INVTMP1}_{(t)} + 1) \wedge \\
 & \quad \quad ((\text{PAIR2}_{(t)} + \text{values}_{i,1}) \cdot \text{INVTMP2}_{(t)} + 1))] \wedge \\
 & (\text{PAIR2}_{(t)} + \text{values}_{i,0}) \cdot (\text{PAIR2}_{(t)} + \text{values}_{i,1}) \cdot [\\
 & \quad \text{LastFlag}_{(t)} \cdot (\text{NewFlag}_{(t)} + 1) \cdot [\\
 & \quad \quad ((\text{PAIR1}_{(t)} + \text{values}_{i,0}) \cdot \text{INVTMP3}_{(t)} + 1) \wedge \\
 & \quad \quad ((\text{PAIR1}_{(t)} + \text{values}_{i,1}) \cdot \text{INVTMP4}_{(t)} + 1))] \wedge \\
 & (\text{NewFlag}_{(t)} + \text{LastFlag}_{(t)}) \cdot [\\
 & \quad ((\text{PAIR1}_{(t)} + \text{values}_{i,1}) \cdot \text{INVTMP4}_{(t)} + 1) \cdot ((\text{PAIR2}_{(t)} + \text{values}_{i,0}) \cdot \text{INVTMP1}_{(t)} + 1) \wedge \\
 & \quad ((\text{PAIR1}_{(t)} + \text{values}_{i,0}) \cdot \text{INVTMP3}_{(t)} + 1) \cdot ((\text{PAIR2}_{(t)} + \text{values}_{i,1}) \cdot \text{INVTMP2}_{(t)} + 1)] \wedge \\
 & (\text{LastFlag}_{(t)} + 1) \cdot (\text{LastFlag}_{(t)} + X) \cdot \text{NewFlag}_{(t)}
 \end{aligned}$$

原論文 Figure 14 : profile pair comparison を実現する AIR constraints。

$$\begin{aligned}
 & \text{InnerMatch}_{(t)} \cdot (\text{InnerMatch}_{(t)} + 1) \cdot (\text{Match}_{(t+1)} + X) \wedge \\
 & \text{Match}_{(t)} \cdot (\text{Match}_{(t)} + 1) \cdot (\text{Match}_{(t+1)} + X) \wedge \\
 & \text{InnerMatch}_{(t)} \cdot (\text{InnerMatch}_{(t)} + 1) \cdot (\text{Match}_{(t)} + X) \cdot (\text{Match}_{(t+1)} + 1) \wedge \\
 & (\text{InnerMatch}_{(t)} + 1) \cdot (\text{InnerMatch}_{(t)} + X) \cdot (\text{Match}_{(t)} + 1) \cdot (\text{Match}_{(t)} + X) \cdot \text{Match}_{(t+1)}
 \end{aligned}$$

原論文 Figure 15 : match result の集計・出力整合性を実現する AIR constraints。

DPM 全体の AIR parameter は width 81、1 entry あたり 62 cycles、degree 8、system size 626、AIR 内 multiplication gates 1467、expanded circuit 換算で 90,954 multiplications/entry。したがって n entries では depth $\approx 62n$ 、multiplication count $\approx 90,954n$ となる。

構成要素	主な処理	AIR 上の特徴
database commitment	各 profile を AES-DM で chain/hash	hash AIR が支配
profile commitment	候補者 profile の commit 確認	固定小入力だが同 hash primitive
pair comparison	20 loci $\times$ unordered allele pair	equality/inverse/selectors
result aggregation	full/partial/no match	counter と Boolean selector
public binding	cmt, cmp, α との一致	boundary constraints

17.6 この用が明しないもの

ZK-STARK は「committed database に対して指定 algorithm を正しく実行した」ことを証明する。database が現実世界の全 forensic records を漏れなく含むこと、各 record の採取が適法であること、候補者 profile の由来が正しいこと、commitment 公開時刻より前に改ざんされていないことは、別の provenance・governance 問題である。

計算完全性とデータ完全性は別

証明系は $C(D)=\alpha$ を保証しても、D が『正しい世界状態』であることまでは自動的に保証しない。実運用には、data ingestion の署名、append-only log、sensor attestation、auditable access control、commitment 更新規則を statement へ含める必要がある。

18. SHA2 AIR と affine routing network (Appendices F–G, pp.80–83)

18.1 SHA2 AIR

Appendix F は SHA2 の AIR 化を概説する。SHA2 は bitwise rotation、choice、majority、mod 2^{32} addition を多用し、 $F_{2^{64}}$ 上では carry 処理が非自然である。結果として width 56、3762 cycles、degree 11、system size 1065、expanded multiplication count 約 2.71×10^6 となり、AES128+DM の約 2.33×10^4 より二桁大きい。

primitive	width w	cycles c	degree d	constraints s	expanded × gates
SHA2	56	3762	11	1065	2,708,640
AES128+DM	62	48	8	327	23,328
Rij160+DM	68	58	8	318	51,678
DPM total/entry	81	62	8	626	90,954

この比較は、proof system における“hash friendliness”が field と arithmetization に依存することを示す。native CPU で高速な hash が proof circuit でも高速とは限らない。後年の Poseidon、Rescue、Griffin 等の algebraic hash はこの問題を別方向から解くが、本論文は AES-NI と binary-field AIR の相性を利用した。

18.2 Back-to-back De Bruijn network

Appendix G は、任意 permutation を routing できる rearrangeable network を combinatorial に記述し、各 edge を affine map として有限体へ埋め込む。De Bruijn/butterfly network を前後に接続することで、N inputs から N outputs への任意 bijection を $O(N \log N)$ switches で実現する。

各 switch は 2×2 の straight/cross 選択であり、control bit と copy constraints だけで検査できる。global permutation equality を直接多項式化するより degree を一定に保てる一方、trace length に $\log N$ factor を加える。

$$(u_0, u_1) \mapsto \begin{cases} (u_0, u_1) & s=0 \\ (u_1, u_0) & s=1 \end{cases}, \quad s(s-1)=0$$

Theorem G.3 は routing 設定を $O(N \log N)$ で計算できることを与える。APR general reduction の prover が $O(T \log T)$ で permutation witness を作れるのはこのためである。

18.3 Part II の総括

原論文の全構成は、計算を trace へ、trace を affine algebra へ、affine algebra を二つの RS proximity 主張へ、proximity 主張を FRI へ、最後に Merkle/Fiat-Shamir で non-interactive argument へ変換する。各段階で『局所性』『低次数』『random linear combination』『誤り訂正距離』のいずれかを利用している。

$$\text{Program} \rightarrow \text{AIR} \rightarrow \text{APR} \rightarrow (f^{(0)}, g^{(0)}) \rightarrow \text{FRI} \times 2 \rightarrow \text{Merkle/Fiat-Shamir}$$

Part III 数学的メカニズムを手で追う

AIR quotient、ALI、FRI、zero knowledge、soundness、knowledge extraction を小例と定量式で再構成する

19. 小さな有限で追う AIR→quotient→FRI

注意：教育用の簡略化

以下は手計算可能性を優先して素体 F_{17} と単純な time domain を使う。原論文は F_{2^n} の affine subspace、cyclic embedding、additive FFT を用いるが、vanishing quotient と FRI folding の論理は同じである。

19.1 Program と execution trace

program $x \leftarrow x^2 + 1$ を 4 回反復し、初期値 $x_0 = 2$ とする。 F_{17} 上では

$$x_0 = 2, \quad x_1 = 5, \quad x_2 = 9, \quad x_3 = 14, \quad x_4 = 10$$

時刻 t	x_t	検算 $x_t^2 + 1 \bmod 17$
0	2	$2^2 + 1 = 5$
1	5	$5^2 + 1 = 26 \equiv 9$
2	9	$9^2 + 1 = 82 \equiv 14$
3	14	$14^2 + 1 = 197 \equiv 10$
4	10	最終 state

AIR は一 register、transition polynomial $P(X, Y) = Y - X^2 - 1$ 、boundary $x_0 = 2, x_4 = 10$ である。trace values を時刻 domain $H = \{0, 1, 2, 3, 4\}$ で interpolate すると

$$\hat{f}(X) = 2 + 5X + 3X^2 + 11X^3 + X^4 \in F_{17}[X]$$

実際に $f(0), \dots, f(4)$ は上表の trace と一致する。

19.2 全 transition を一つの divisibility へ

transition を検査する時刻 $S = \{0, 1, 2, 3\}$ の vanishing polynomial は

$$Z_S(X) = X(X-1)(X-2)(X-3) = X^4 + 11X^3 + 11X^2 + 11X$$

transition numerator を $N(X) = f(X+1) - f(X)^2 - 1$ と置く。正しい trace なので $N(t) = 0$ for $t \in S$ 、従って $Z_S | N$ である。実際、 F_{17} 上で

$$N(X) = 16X^8 + 12X^7 + 9X^6 + 9X^5 + 14X^4 + 9X^3 + 5X^2 + 11X$$

$$Q(X) = \frac{N(X)}{Z_S(X)} = 1 + X + 5X^2 + 6X^3 + 16X^4$$

となり余りは 0。verifier が全 4 transition を読む代わりに、prover が Q の low-degree encoding を commit し、random point r で $N(r) = Z_S(r)Q(r)$ を検査すればよい。多項式 identity が偽なら、degree bound D に対し偶然一致する確率は高々 $D/|F|$ （単独の Schwartz–Zippel 型評価）である。

STARK の基本変換

多数の local constraints → vanishing polynomial で割り切れる global numerator → quotient の低次数性。低次数性そのものを FRI で検査するため、verifier は全 trace を読まない。

19.3 複数 register ・ 複数 constraint の random mixing

現実の AIR には w registers と s constraints がある。個別に $Q_{\{j\}}, Q_{\{i\}}^{\text{cmp}}$ を検査すると cost が $w+s$ 倍になる。ALI では verifier が random coefficients a_j, b_i を選び、

$$F(X) = R_F(X) + \sum_{j=1}^w a_j f_j(X), \quad G(X) = R_G(X) + \sum_{i=1}^s b_i Q_i^{\text{cmp}}(X)$$

だけを FRI へ渡す。ある不正 component が残っていると、相殺に必要な係数関係は非自明な linear equation となり、uniform random challenge に対して確率 $\approx 1/|F|$ でしか起こらない。interleaved code の random projection と同じ発想である。

19.4 FRI folding を一段だけ手計算

F_17 の multiplicative group は order 16 であり、 $g=9$ は order 8 を持つ。 $H = \langle 9 \rangle$ を evaluation domain とし、 $\text{degree} \leq 3$ polynomial

$$p(X) = 3 + 2X + 5X^2 + X^3$$

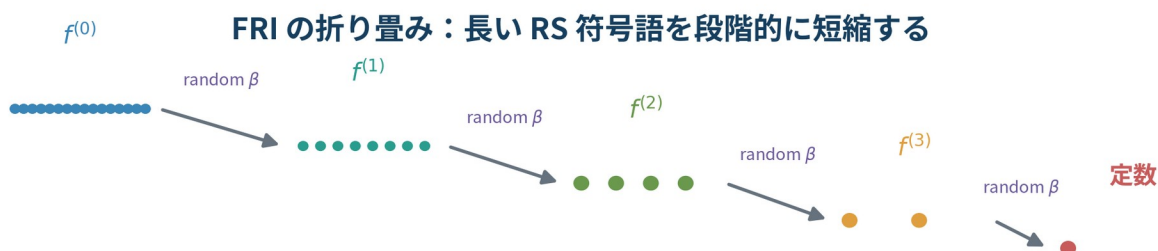
を考える。even/odd decomposition は $p(X) = p_e(X^2) + X p_o(X^2)$ 、 $p_e(Y) = 3 + 5Y$ 、 $p_o(Y) = 2 + Y$ 。verifier challenge $\beta = 7$ に対する fold は

$$p'(Y) = p_e(Y) + \beta p_o(Y) = 12Y \pmod{17}$$

degree は 3 から 1 へ下がる。oracle 値だけで folded value を計算する式は、 x と $-x$ の pair に対して

$$p'(x^2) = \frac{p(x) + p(-x)}{2} + \beta \frac{p(x) - p(-x)}{2x}$$

例えば $x=1$ では $p(1)=11$ 、 $p(-1)=5$ 。右辺は $8+7 \cdot 3=29 \equiv 12$ で、 $p'(1)=12$ と一致する。各 round で domain を $x \mapsto x^2$ により半減し、degree bound も半減する。 $\log|H|$ rounds 後には constant-size polynomial となり、verifier が直接確認できる。



各ラウンドは小さな coset の値をランダム係数で混ぜ、次数上限をほぼ半分にする。

最後の低次数多項式を明示確認し、QUERY では全レベルの整合性を少数点で辿る。

図 19-1 : FRI folding。pair $(x, -x)$ の二評価から challenge β で次 layer の一評価を作り、domain と degree bound を同時に縮小する。

19.5 なぜ不正高次数 word が fold で露見するか

honest low-degree polynomial は every round で正しい degree-halving relation を持つ。任意 word でも fold 値自体は作れるが、複数 round を通じて一貫して小 degree へ落ちるには、元 word が RS codeword へ近い必要がある。FRI soundness proof は、random fold が distance を高確率で保存することと、最後の small polynomial check を組み合わせる。

ここで検査対象は『polynomial identity が random point で成立するか』ではなく、『巨大 evaluation vector が低次数 code へ近いのか』である。FRI は誤り訂正 code の proximity test であり、単純な一点 Schwartz-Zippel 検査より強い。

20. Zero knowledge の代数的構成

20.1 Randomized low-degree extension

secret vector $z: H \rightarrow F$ を一意な $\text{degree} < |H|$ polynomial で encode すると、domain 外の少数 query でも z に関する linear information を漏らす可能性がある。そこで degree を b だけ増やし、 H 上の値を固定した random polynomial を選ぶ。

$$\bar{f}(X) = \hat{f}_0(X) + Z_H(X)R(X), \quad \deg R < b$$

H 上では $Z_H = 0$ なので $\bar{f}|_H = z$ を保つ。一方 $L \cap H = \emptyset$ なら $Z_H(a) \neq 0$ for $a \in L$ であり、任意 b 個以下の distinct evaluations は R の自由係数により一様独立になる。これが bounded independence である。

完全 ZK の構造 : bounded independence と self-reducibility



各評価値をランダムに見せるだけでは除算・合成で乱数が消える。ランダム符号語を混ぜた主張へ自己還元し、検査対象自体をランダム化する。

図 20-1 : 秘密 trace を固定したまま、 $\text{vanishing polynomial} \times \text{random polynomial}$ を加えて LDE を randomize する。secret domain H と query domain L を disjoint に取る。

20.2 なぜ bounded independence だけでは足りないか

quotient や sumcheck のような後続変換は、入力 randomness を消してしまうことがある。例えば f が H 上で 0 なら、 $f = Z_H R$ の quotient は R となり、protocol message 間の相関から情報が現れる可能性がある。したがって各 subprotocol の出力分布まで含めて simulation 可能にする必要がある。

20.3 Random self-reduction

prover は random RS codeword r と、その H 上の $\text{sum } \beta = \sum_{a \in H} r(a)$ を先に commit する。verifier challenge c 後、元の claim $\Sigma f = \mu$ を、randomized claim $\Sigma(cf+r) = c\mu + \beta$ へ変換する。

$$\sum_{a \in H} (c f(a) + r(a)) = c\mu + \beta$$

r が uniform なので $cf+r$ も uniform (c 固定後) であり、後続 sumcheck/FRI transcript は witness に依存しない分布として simulate できる。元 claim が偽なら、random c で偶然新 claim が真になる確率は $1/|F|$ 。Aurora の univariate sumcheck ZK も同じ構造を用いる。

20.4 Query-bounded perfect ZK から computational ZK へ

IOP 段階では、malicious verifier が各 oracle へ行う distinct query 数 b を上限として perfect simulation を証明する。Merkle/Fiat-Shamir 変換後、hash を random oracle として扱うことで non-interactive computational ZK へ移す。実装では b を honest verifier の query 数以上に設定し、masking degree $\cdot$ domain size を調整する。

この ZK は『oracle 全体を公開しない』IOP model に依存する。Merkle root は binding commitment であり、verifier は少数 leaf だけを開く。全 LDE を公開すれば、randomized polynomial でも degree 以上の点から秘密部分を復元できる。

21. Soundness error の分解と積算

STARK の soundness は一つの仮定で決まらず、複数層の failure probability の和として評価する。union bound による保守的な積算を明示すると、security parameter 設計が透明になる。

$$\epsilon_{\text{total}} \leq \epsilon_{\text{arith}} + \epsilon_{\text{mix}} + \epsilon_{\text{FRI,commit}} + \epsilon_{\text{FRI,query}} + \epsilon_{\text{hash}} + \epsilon_{\text{FS}}$$

項	失敗事象	典型的依存
ϵ_{arith}	不正 constraint が field identity で偶然消える	$\text{degree}/ F $
ϵ_{mix}	random linear combination で不正成分が相殺	$1/ F $ または反復
$\epsilon_{\text{FRI,commit}}$	fold challenges が bad transcript を選ぶ	$N/ F $ 型 + 分析定数
$\epsilon_{\text{FRI,query}}$	query が error positions を外す	$(1-\delta)^\ell$ 型
ϵ_{hash}	Merkle binding 破壊 / collision	digest length, hash 仮定
ϵ_{FS}	Fiat-Shamir programming / QROM loss	ROM/QROM reduction

原論文 Lemma B.6 の proved analysis では、ALI error、FRI COMMIT error、FRI QUERY error を各 $2^{-(\lambda+\text{const})}$ 程度へ割り当て、総和 $\leq 2^{-\lambda}$ とする。query repetition ℓ は

$$\ell = \left\lceil \frac{\lambda+2}{\log_2(1/(1-\delta/10))} \right\rceil$$

のように distance $\delta/10$ へ依存する。distance が小さいほど ℓ が増えるため、ALI の距離損失 ζ が proof size へ直接効く。

21.1 60-bit prototype の意味

DPM prototype は $\lambda=60$ 、 $F_{2^{64}}$ 、AES-DM collision security 推定 64 bit。これは当時の proof-of-concept としては合理的だが、長期保護・金融用途・quantum query を考える現代の 128-bit target とは異なる。

hash output が 128 bit なら classical collision security は約 64 bit (birthday bound)。paper の AES128-DM Merkle もこの理由で 64-bit collision estimate となる。128-bit collision security には通常 256-bit digest が必要である。

“post-quantum”の限定

本構成は pairing/DLP/RSA を使わず、主に hash と情報理論的 IOP へ依存するため、既知の Shor 型攻撃には強い。しかし量子 adversary は Grover/BHT 型 speedup、QROM での Fiat-Shamir reduction loss、digest length に影響する。したがって『plausibly post-quantum』であり、無条件な量子安全性を意味しない。

21.2 Proven bound と conjectured bound

対象	proved analysis	prototype での扱い
ALI distance	$\delta/(2\zeta)$ 以上のどちらか一方	Conj. B.17 で code distance 近くを期待
FRI query rejection	δ_0 以下で明示 bound	Conj. B.19 で blockwise distance に近いと期待
field-size error	$1/ F $ 項を明示	$F_{2^{64}}$ で約 2^{-64}
Merkle binding	collision-resistant hash 仮定	AES128-DM、約 64-bit collision
non-interactivity	random oracle model	concrete hash で heuristic 実現

22. Proof of knowledge と extractor

22.1 Soundness との違い

soundness は false statement を accept しにくいことをいう。proof of knowledge はさらに、accept probability が knowledge error を上回る prover から、実 witness を効率的に抽出できることを要求する。DPM では commitment を開く D', p' と valid trace が対象になる。

22.2 抽出 chain

1. FRI extractor/decoder により、accepting oracle を近傍 RS codeword へ補正する。
2. ALI extractor は、 $f^*(0), g^*(0)$ が双方十分近いことから、各 APR assignment codeword を復元し、constraints を満たす APR witness を得る。
3. APR→AIR extractor は、 $Q_j = w_j Z_{\{B,j\}} + E_{\{B,j\}}$ を time domain で評価し execution trace を回復する。
4. Cook-Levin 型 reduction の逆写像により、元計算の nondeterministic witness または secret input を取り出す。

extractor は通常 black-box access、rewinding、複数 transcript、RS decoding を用いる理論 algorithm であり、実際の system が証明ごとに witness を抽出するわけではない。security definition として『説得力のある prover は witness を内在させる』ことを保証する。

22.3 Commitment binding との関係

ZK-STIK 段階の knowledge は information-theoretic だが、Merkle 化した STARK では prover が複数の異なる oracle を同じ root へ開けないという hash binding が必要となる。collision が作れれば extractor が見る oracle が一意でなくなり、argument of knowledge が崩れる。

23. 複雑度・parameter・実装 cost の再計算

23.1 Theorem 3.4 と 3.5

量	space-bounded T,S	general T
prover arithmetic	$O(S T \log T)$	$O(T \log^2 T)$
verifier arithmetic	$O(\lambda(S + \log T))$	$O(\lambda \log T)$
proof oracle length	$O(TS)$	$O(T \log T)$
rounds	$\frac{1}{2} \log T + O(1)$	$\frac{1}{2} \log T + O(1)$
queries	$O(\lambda(S + \log T))$	$O(\lambda \log T)$
ZK の強さ	$T = \text{poly}(n)$ で perfect ZK	$T = \text{poly}(n)$ で perfect ZK

proof oracle length は IOP 内部の全 message 長であり、最終 STARK argument size ではない。Merkle commit 後は root、query values、authentication paths だけを送るため polylogarithmic になる。ただし prover は全 oracle を生成・commit するので linear/quasi-linear 作業を負う。

23.2 DPM の規模換算

1 profile あたり 40 bytes、62 cycles、90,954 multiplication gates 相当。 $n = 2^{20} \approx 1.05 \times 10^6$ なら

$$|D| = 40 \cdot 2^{20} \approx 40 \text{ MiB}$$

$$T_{\text{cycles}} = 62 \cdot 2^{20} \approx 6.50 \times 10^7$$

$$N_{\text{mult}} = 90,954 \cdot 2^{20} \approx 9.54 \times 10^{10} \approx 2^{36.47}$$

paper 本文の約 43 MB は decimal MB 換算と metadata/padding 差を含む表現と理解できる。この規模で proof generation が可能だった一方、理論 oracle 総量は RAM を大幅に越えるため再計算が必要になった。

23.3 Communication formula

$$CC = q_{\text{total}} \log_2 |F| + AP_{\text{total}} \lambda_h$$

第一項は開示する field elements、第二項は Merkle authentication nodes。 λ_h は hash digest bits。row packing と coset packing は AP_{total} を減らす、一 leaf payload が大きくなる。optimal arity、leaf width、query clustering は hardware/network と security に依存する。

設計変数	増やすと	減らすと
RS blowup / inverse rate R	distance ↑、oracle/Merkle ↑	proof ↑ を抑えるが soundness ↓
query repetitions ℓ	soundness ↑、openings ↑	proof 短縮、error ↑
field size	algebraic error ↓、element bytes ↑	compact だが $1/ F $ 項 ↑
ZK degree b/k	query privacy ↑、rate ↑	proof 軽量だが ZK bound 低下
Merkle digest	binding ↑、path bytes ↑	compact だが collision margin ↓
FRI folding factor	rounds ↓、per-round work/analysis 変化	rounds ↑、simple

24. 批判的読解：主張・限界・再現性

24.1 論文が確立したこと

- 透明 setup、hash-based non-interactivity、plausible post-quantum security を同時に備える汎用 ZK-STARK を実装した。
- sequential で deep/narrow な DPM に対し、verification time と communication が native computation/witness より小さくなる具体 threshold を示した。
- register-wise encoding と ALI により composition degree を大幅に削減し、FRI と additive FFT で quasi-linear prover を実現した。
- AIR→APR→ALI→FRI→Merkle という、後続 STARK/zkVM 設計の原型を明確化した。

24.2 論文が確立していないこと

- 128-bit security での end-to-end 性能。prototype は 60-bit target と約 64-bit hash collision estimate である。
- 全 benchmark 点での proof generation。巨大 n の verifier 測定には synthetic proof が含まれる。
- DPM database の現実世界における完全性・適法性・provenance。
- conjectured ALI/FRI soundness を使わない同等の concrete query 数。
- すべての computation topology で他方式より優れること。small、parallel、repeated fixed circuit では別方式が有利になりうる。

24.3 再現時のチェックリスト

1. paper version と code commit を固定し、security estimator の proved/conjectured toggle を確認する。
2. field representation、irreducible polynomial、FFT basis、subspace polynomial を記録する。
3. AIR compiler が出す width/cycles/degree/constraint count を独立検算する。
4. Merkle hash、digest length、tree arity、leaf packing、path pruning を固定する。
5. native execution、prover、verifier、serialization、network を分離測定する。
6. peak RSS、memory traffic、disk spill、recomputation を含む time-space profile を報告する。
7. false statements/corrupted oracle に対する negative tests と fuzzing を行う。
8. Fiat-Shamir transcript domain separation と canonical serialization を監査する。

24.4 原文上の注意点

箇所	原文	本書の判断
Section 2.3	degree improvement $6.5 \times 10^4 - 1.8 \times 10^5$	Figure 5 と式では $6.5 \times 10^3 - 1.85 \times 10^4$ 。本文の一桁誤記と判断
Theorem 3.5 冒頭	$\text{err}(n) = 2^{\lambda(n)}$ と読める	直後の $\leq 2^{-\lambda}$ と証明から負指数が正しい
巨大 verifier 測定	$n = 2^{36}$ 等	non-adaptive verifier + random/synthetic proofs。end-to-end ではない
post-quantum	hash-based	plausible。QROM と digest margin を別途評価

Part IV Aurora および他の汎用 ZKP との比較

計算モデル、arithmetization、commitment、setup、量子耐性、再帰、proof size、prover/verifier の設計空間

25. 2018 ZK-STARK と Aurora の連続性・相違

2018 STARK と 2019 Aurora：共通基盤と最適化対象

STARK / AIR		Aurora / R1CS	
入力モデル	プログラム+時間上限	入力モデル	明示的 R1CS 行列
中間関係	AIR → APR	中間関係	rowcheck + lincheck
代数検査	ALI → 2つの RPT	代数検査	単変数 sumcheck
LDT	FRI	LDT	FRI を 1本に集約
適性	巨大・逐次計算 uniform computation	適性	明示的回路 non-uniform computation

共通点：透明セットアップ、RS 符号、公開コイン IOP、FRI、Merkle/Fiat-Shamir、ハッシュ中心の耐量子性

図 25-1：2018 ZK-STARK と 2019 Aurora。共通基盤は RS code + FRI + IOP + Merkle/Fiat-Shamir。差は、前者が program execution/AIR、後者が explicit R1CS を直接扱う点にある。

25.1 共通する設計 DNA

- public-coin IOP を random oracle/Merkle で non-interactive argument へ変換する。
- witness を Reed-Solomon codeword として扱い、low-degree proximity を FRI で検査する。
- 複数 oracle/constraints を random linear combination で一つの proximity test へ batch する。
- vanishing polynomial と quotient により、局所 constraint の全点成立を低次数性へ変換する。
- randomized LDE と masking codeword で bounded-query perfect ZK を作り、compiler 後に computational ZK を得る。
- 透明 setup、symmetric cryptography、plausible post-quantum security を重視する。

25.2 計算モデル：uniform program 対 explicit constraints

軸	2018 ZK-STARK	Aurora
native relation	AIR/APR：program + time/space bound	R1CS：A,B,C matrices + public input
uniformity	同一 program を長時間実行する uniform computation に自然	各 constraint を明示する non-uniform computation に自然
memory	APR routing で RAM consistency を処理	R1CS compiler が memory/control-flow を展開
verifier input	program 記述は小さく、T に polylog	明示 R1CS を読むため N に線形
典型対象	VM、trace、streaming、長い逐次計算	既存 R1CS circuit、固定回路、field arithmetic

同じ N-gate explicit circuit を比較するなら、STARK はその circuit を program として走らせるため routing overhead を負い、Aurora は R1CS を直接扱う。逆に program P が小さく実行時間 T が巨大なら、STARK verifier は $|P| + \text{polylog} T$ で済むが、Aurora verifier は展開された R1CS 全体を読む必要がある。

25.3 Arithmetization：ALI 対 univariate sumcheck

ZK-STARK は AIR constraints を APR へ配置し、ALI で assignment/composition の二 RPT へ落とす。Aurora は R1CS の条件 $(Az) \cdot (Bz) = Cz$ を、三つの lincheck と一つの rowcheck へ分解する。

$$\text{R1CS: } \bigwedge (Az) \cdot (Bz) = Cz$$

lincheck は random α で線形関係を一つの sum claim へ圧縮し、Aurora 独自の univariate sumcheck で検査する。rowcheck は $(f_A \alpha - f_B - f_C)/Z_H$ が低次数であることを直接確認する。したがって routing を回避できる。

サブ問題	ZK-STARK	Aurora
local transition/product	composition quotient over AIR	rowcheck
linear map	register neighbor maps/APR	lincheck
多数 constraint の batch	ALI random combination	amortized univariate sumcheck
low-degree enforcement	FRI×2	combined FRI (概ね一系統)
最大 oracle 構造	register 数・neighbor 数に依存	5 主要 oracles+sumcheck auxiliaries

25.4 Univariate sumcheck の意義

classical sumcheck は multilinear/tensor structure を用い、変数一つずつ消す。Aurora は univariate RS polynomial f について $\sum_{a \in H} f(a) = \mu$ を証明する。affine subspace H では $\deg f < |H| - 1$ の多項式の sum が 0 になるため、 $f = g + \beta X^{|H|-1} + Z_H h$ と分解し、 β を target sum で固定する。

$$\hat{f}(X) = \hat{g}(X) + \beta X^{|H|-1} + Z_H(X) \hat{h}(X), \quad \sum_{a \in H} \hat{f}(a) = \beta \sum_{a \in H} a^{|H|-1}$$

この手法により、proof oracle length $O(N)$ 、query $O(\log N)$ 、prover $O(N \log N)$ の R1CS IOP を得る。2018 STARK の general routing と比べ、explicit R1CS では一つ log factor と大きな定数を削減する。

25.5 漸近比較

量 (explicit N-constraint R1CS)	2018 STARK を R1CS へ適用	Aurora
IOP proof length	$O(N \log N)$	$O(N)$
IOP queries	$O(\log N)$	$O(\log N)$
prover	$O(N \log^2 N)$	$O(N \log N)$
verifier	$O(N)$ (R1CS 読取り込み)	$O(N)$
最終 argument	$O(\log^2 N)$	$O(\log^2 N)$
主な追加 overhead	routing、二 FRI、APR registers	lincheck/sumcheck、R1CS matrices

ここで最終 argument $O(\log^2 N)$ は、 $O(\log N)$ queries それぞれに $O(\log N)$ Merkle path が付くため。column hashing/path pruning で定数は下がるが、基本構造は同じである。

25.6 Concrete comparison の読み方

Aurora paper は 128-bit target、 $F_{2^{192}}$ 、最大 2^{20} constraints で proof 約 40–130 kB、proving は fraction of a second から minutes、verification は milliseconds から seconds と報告する。witness より proof が小さくなる threshold は約 $m \geq 2000$ 。

同 paper の R1CS benchmark では Aurora proof は Ligero より漸近的に小さく、R1CS を APR で simulated した Stark より大幅に小さい。一方、2018 DPM STARK は uniform sequential computation としての verifier compression を主目的とし、security target、field、relation、hardware が異なるため、絶対時間の直接比較は不適切である。

結論

Aurora は STARK を否定するのではなく、同じ IOP/FRI 技術を explicit R1CS へ特化し、routing を univariate sumcheck で置き換えた。program が succinct なら AIR/STARK、constraint system が既に明示されるなら Aurora 型、という計算モデル選択が本質である。

26. Pairing-based SNARK : Groth16 • PLONK • Marlin

26.1 Groth16

Groth16 は R1CS/QAP を pairing group へ commit し、proof を 3 group elements、verification を 3 pairings を含む一つの pairing product equation へ圧縮する preprocessing zkSNARK である。proof size と on-chain verification は極めて小さい。

強み	弱み
proof が constant size (3 group elements)	circuit-specific trusted setup が必要
verification が非常に軽い	pairing/DLP に依存し post-quantum ではない
固定回路で大量 proof を作る用途に強い	setup toxic waste、ceremony、key 配布・更新が運用負担
成熟した R1CS toolchain と相性がよい	recursion には curve cycle / non-native arithmetic の工夫が必要

2018 STARK と比べると、proof/verification は Groth16 が圧倒的に小さくなりやすいが、transparency と PQ plausibility を失う。また、proving key が circuit size に比例して大きく、回路変更時の setup 問題がある。

26.2 PLONK 系

PLONK は Lagrange basis 上の permutation argument を使い、universal/updatable SRS で多数回路を扱う。wiring を copy constraints として grand product へまとめ、custom gates と lookup を拡張しやすい。

Groth16 の per-circuit setup を緩和し、Plonkish arithmetization は実装エコシステムの中心となった。ただし KZG commitment を使う典型構成は pairing/SRS 依存で非 PQ。IPA commitment へ置換すれば透明化できるが、proof/verification profile が変わる。

26.3 Marlin

Marlin は R1CS に対する Algebraic Holographic Proof と polynomial commitment を組み合わせ、universal/updatable SRS、constant-size argument を実現する。Aurora と同様 R1CS を対象にするが、code-based IOP ではなく algebraic polynomial commitment へ低次数主張を外部化する。

26.4 STARK/Aurora との比較

軸	Groth16/PLONK/Marlin	STARK/Aurora
setup	structured SRS (方式により per-circuit/universal)	transparent public randomness
仮定	pairing/DLP/knowledge assumptions	hash + ROM + IOP
PQ	通常 No	plausibly Yes
proof size	数百 B ~ 数 kB 級が典型	数十 kB ~ MB 級が典型
verification	pairings/MSM で小さい	hash + field ops + Merkle paths
prover	FFT + MSM、GPU 適性	FFT/field + hash、memory bandwidth 支配
回路表現	R1CS/Plonkish	AIR または R1CS

“trusted setup なし”だけで選ぶと STARK が有利だが、proof bandwidth や L1 gas が最重要なら pairing SNARK が有利になりやすい。実用 system では、STARK で大計算を証明し、その verifier を small pairing SNARK で wrap する hybrid も成立する。

27. DLOG/IPA • sumcheck • folding 系

27.1 Bulletproofs

Bulletproofs は Pedersen commitment と recursive inner-product argument を用い、transparent setup で logarithmic proof を得る。range proof や中規模 arithmetic circuit に適し、proof aggregation も可能。一方、general circuit verification は多数の group operations を要し、DLP に依存するため post-quantum ではない。

27.2 Halo/Halo2

Halo は DLP-based polynomial commitment と accumulation により、trusted setup なしの practical recursive proof composition を示した。Halo2 系は Plonkish arithmetization、custom gates、lookup、IPA または KZG 等の commitment backend を組み合わせる。recursion と circuit ergonomics に強いが、IPA 版は DLP 依存、KZG 版は SRS/pairing 依存である。

27.3 Spartan

Spartan は RICS を multilinear extension へ写し、classical sumcheck と multilinear polynomial commitment を組み合わせる。time-optimal prover を狙い、statement preprocessing/computation commitment により sublinear verifier も構成できる。透明な DLOG 版は trusted setup を不要にするが、量子耐性はない。

Aurora が univariate RS code + FRI で sumcheck 類似問題を解くのに対し、Spartan は Boolean hypercube 上の multilinear polynomial を直接 sumcheck する。前者は hash/PQ、後者は group commitment と multilinear algebra の効率が主な性格差である。

27.4 Nova ◻ HyperNova

Nova は各 step の RICS instance を relaxed RICS として fold し、二つの instance を一つへ蓄積する。毎 step で full SNARK を作らず、constant recursion overhead で IVC を進め、最後だけ succinct SNARK で compress できる。

HyperNova は Customizable Constraint Systems (CCS) へ folding を拡張し、Plonkish/RICS/AIR を統合的に扱う。2026 年改訂版では NovaBlindFold として folding-based ZK を整理している。a-la-carte instruction cost と multi-folding は zkVM/PCD に適する。

軸	STARK	Nova/HyperNova
基本 primitive	IOP + proximity test	folding + vector/polynomial commitment
長い計算	全 trace を一括 LDE/FRI	step ごとに incremental accumulation
prover memory	大 oracle/Merkle が重い	streaming/online にしやすい
final proof	そのまま polylog、または再帰集約	通常 compressing SNARK を追加
PQ	hash 版は plausible	典型 DLOG commitment は No
適性	batch large trace、parallel proving	IVC、stateful computation、proof-carrying data

両者は競合というより補完的である。folding で長い step 列を蓄積し、hash-based SNARK/STARK で最終 compress する研究方向も考えられる。

28. Hash/code-based 後 : Ligerio • Fractal • Brakedown

28.1 Ligerio

Ligerio は MPC-in-the-head/linear code 的な interleaved encoding を用い、transparent • plausibly PQ な argument を構成する。round 数が少なく実装が比較的単純だが、query/proof が $O(\sqrt{N})$ で、 N が大きいほど Aurora の $O(\log^2 N)$ argument に劣る。small/medium circuit や batching では定数面で有利な場合がある。

28.2 Fractal

Fractal は holographic preprocessing を用いて、transparent • post-quantum な recursive SNARK を R1CS 向けに構成する。statement を preprocess/encode し、再帰 verifier を効率化する。Aurora の R1CS/FRI 系統を recursive composition へ伸ばす位置付けである。

28.3 Brakedown / Shockwave

Brakedown は linear-time encodable code と Spartan 系 linear-time IP を組み合わせ、R1CS に対し $O(N)$ field operations の prover を実装した transparent • plausibly PQ SNARK である。field-agnostic で、FFT-friendly field への拘束を緩める。

代償は code choice と proof/verifier trade-off。RS code を用いる Shockwave variant は線形時間性を失う代わりに proof と verification を短くする。これは『FRI/RS の優れた distance • succinctness』と『linear-time encodable code の prover 速度』のトレードオフを明示する。

28.4 AIR/STARK との差

Brakedown 等は non-uniform R1CS を対象にし、2018 STARK の uniform program succinctness は持たない。逆に R1CS が明示されるなら routing を避け、prover を線形化できる。計算モデル選択がここでも中心である。

29. FRI 以後の proximity testing と small-field 化

29.1 DEEP-FRI / DEEP-ALI

DEEP は evaluation domain 外の random point z で interpolating polynomial の値を要求し、複数の近傍 codeword 候補 (pretenders) を一つへ固定する。DEEP-FRI は FRI の soundness を list-decoding radius 近くまで改善し、DEEP-ALI は ALI の soundness gap を小定数から 1 に近い値へ高める。

2018 論文で conjecture に頼った距離保存問題へ、より強い protocol design で対応したと理解できる。

29.2 STIR

STIR (Shift To Improve Rate) は round ごとに RS code rate を改善し、 λ -bit security で query complexity を $O(\log d + \lambda \log \log d)$ へ抑える。FRI の $O(\lambda \log d)$ から、security repetition と degree recursion の掛け算を緩和する。論文実装は FRI より 1.25–2.46 倍小さい argument を報告する。

29.3 WHIR

WHIR は constrained Reed–Solomon code の IOPP として、low query と非常に高速な verification を狙う。multilinear/univariate polynomial evaluation claim を近接検査へ直接取り込み、FRI/STIR 等の置換候補となる。2024/2025 論文は数百 microseconds 級 verifier の例を報告するが、parameter・implementation 条件を揃えた比較が必要である。

29.4 Binius / binary-field towers

Binius 系は tower of binary fields 上で multilinear polynomial commitment と Plonkish/lookup を構成し、bitwise hash (Keccak 等) を大きな prime field へ embedding する overhead を避ける。2018 STARK も binary field を選んだが、主に additive FFT と AES-friendly AIR が理由だった。Binius は small-field arithmetization を multilinear/sumcheck/commitment 側から再設計する。

29.5 Circle STARKs

traditional STARK は field 内に smooth-order multiplicative subgroup を必要とする。Circle STARKs は circle curve $x^2 + y^2 = 1$ 上の group と FFT を使い、 $p+1$ が 2-smooth な Mersenne-type prime 等で efficient STARK を可能にする。field selection の自由度と machine-word arithmetic を改善する方向である。

系統	改善対象	2018 FRI/STARK に対する意味
DEEP-FRI/ALI	soundness gap	conjecture 依存・query 回数を削減
STIR	query complexity	$\lambda \log d$ を $\log d + \lambda \log \log d$ へ
WHIR	verifier time + rich constraints	LDT をより強い PCS/IOPP へ統合
Binius	tiny/binary field	bitwise computation の embedding overhead を削減
Circle STARKs	FFT domain/field choice	smooth subgroup 制約を circle domain で緩和
Brakedown	prover linear time	RS/FFT 以外の code design を活用

30. 汎用 ZKP の選択表

汎用 ZKP の設計空間（定性的）

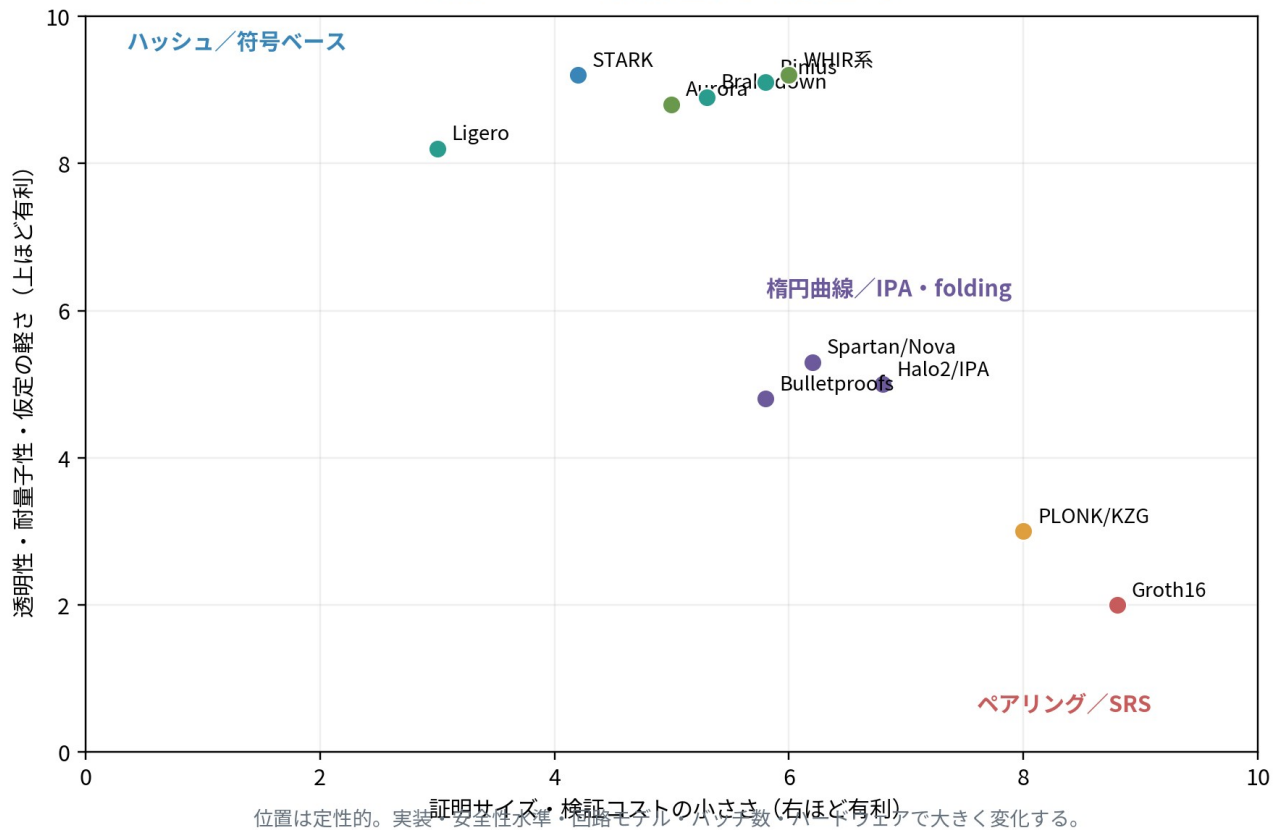


図 30-1：汎用 ZKP の設計空間。proof size、setup、PQ、recursion、uniformity、prover memory のどれを優先するかで選択が変わる。

方式	setup	PQ	native relation	proof	verifier	主な強み	主な弱み
Groth16	circuit SRS	No	R1CS/QAP	constant	very small	最小級 proof/verify	per-circuit ceremony
PLONK/KZG	universal SRS	No	Plonkish	small	small	custom gate/lookup	SRS・pairing
Halo2/IPA	transparent	No	Plonkish	log-ish	MSM-heavy	recursion/ergonomics	DLP、PQ 不可
Spartan	transparent*	No*	R1CS/ML	polylog-sublinear	preprocess 依存	time-optimal 系	commitment 依存
Nova/HyperNova	transparent*	No*	R1CS/CCS	accumulator + compress	incremental	IVC/streaming	final compression 必要
Aurora	transparent	Yes †	R1CS	$O(\log^2 N)$	$O(N)$	hash/PQ、routing 不要	明示 R1CS 読取り
2018 STARK	transparent	Yes †	AIR/APR	polylog	polylog T	uniform long trace	proof/hash/memory 大
Ligero	transparent	Yes †	R1CS/circuit	$O(\sqrt{N})$	$O(N)$	単純・small N	large N proof
Brakedown	transparent	Yes †	R1CS	sublinear	sublinear	linear prover	code 定数/証明長
Binius/WHIR 系	transparent	Yes †	ML/Plonkish 等	polylog	fast	small-field/新 LDT	新規・parameter 複雑

* commitment backend に依存。† hash-based かつ ROM/QROM を適切に扱う意味で plausibly post-quantum。

30.1 要件別の推奨

最優先要件	第一候補	理由	注意
最小 proof/on-chain verify	Groth16 または KZG-Plonkish	pairing による constant/small proof	SRS・非 PQ

最優先要件	第一候補	理由	注意
透明+PQ+長い VM trace	STARK/AIR	uniform program、polylog verifier in T	prover memory、proof bandwidth
透明+PQ+explicit RICS	Aurora/Brakedown/WHIR 系	routing 不要、hash-based	verifier relation read
逐次 IVC／online	Nova/HyperNova	step folding、low memory	DLP と final compression
custom gates/lookups	Plonkish/Halo2/Binius	constraint ergonomics	backend で setup/PQ が変化
tiny-field/bitwise workload	Binius/Circle/modern STARK	embedding overhead 削減	tooling maturity
small-medium circuit／簡潔実装	Ligero/Bulletproofs	低 round／実装単純性	asymptotic verifier/proof

一つの“最強 ZKP”は存在しない

比較すべき単位は protocol 名ではなく、(arithmetization, polynomial/code commitment, low-degree/sumcheck, Fiat-Shamir, recursion, hardware)の組である。同じ PLONK でも KZG と IPA では setup/PQ/verification が異なり、同じ STARK でも field、FRI variant、hash、AIR compiler で性能が大きく変わる。

Part V 応用・システム設計・研究課題

blockchain、zkVM、confidential audit、科学計算、AI、data provenance、再帰、実装監査

31. Blockchain scalability と validity proof

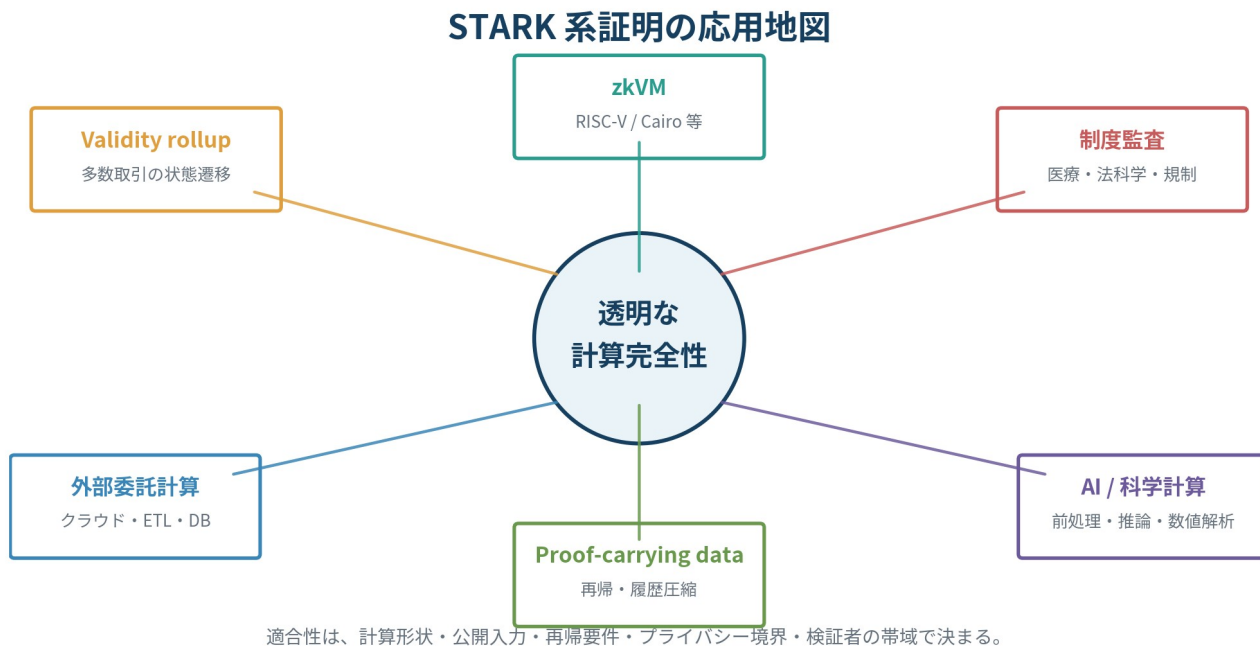


図 31-1 : ZK-STARK/Aurora 型 proof の応用地図。計算完全性を、ledger、VM、database、scientific pipeline、AI inference へ接続する。

31.1 Validity rollup の基本形

sequencer/prover が多数 transaction を実行し、old state root、transaction batch、new state root の関係を proof する。L1 verifier は全 transaction を再実行せず、proof と必要な public data だけを検査する。

$$\text{Verify}(\text{root}_{\text{old}}, \backslash, \text{batchCommit}, \backslash, \text{root}_{\text{new}}, \backslash, \backslash \pi) = 1$$

2018 論文の『一 prover node が ledger state transition を quasi-linear に証明し、他 node が polylog time で検証する』という構想は、validity rollup の抽象形そのものである。AIR は VM execution trace へ自然に対応し、program が固定なら verifier input を小さく保てる。

31.2 Computational validity と data availability

proof は state transition が正しいことを示すが、users が state を再構成するための transaction data が利用可能であることは別問題である。data availability sampling、erasure coding、L1 calldata/blob、committee 等を別途設計する。

性質	ZK/STARK proof が保証	別 mechanism が必要
execution validity	Yes	—
state root binding	Yes (hash 仮定)	—
data availability	No	DA layer / sampling / publication
censorship resistance	No	inclusion rules / forced transactions
sequencer liveness	No	fallback / multi-sequencer
economic finality	No	consensus

31.3 Proof aggregation と recursion

batch を小 proof へまとめても、長期 ledger では proof 数が増える。recursive STARK、Fractal、folding、pairing wrapper を使い、複数 proof を一つに aggregation する。

$$\backslash \pi_{t+1} = \text{Prove} \bigl(\text{Verify}(\backslash \pi_t) = 1 \backslash \wedge \backslash \text{Transition}_{t=1} \bigr)$$

hash-based STARK の verifier を同じ AIR で証明する場合、hash/arithmetic field mismatch と Merkle verification cost が支配する。外側に pairing SNARK を置けば final proof を極小化できるが、PQ/transparency の end-to-end 性は外側の仮定へ低下する。

31.4 Privacy

transaction details、balances、identity credentials を witness とし、state transition の規則だけを publicly verify できる。ただし metadata、timing、public nullifier、state access pattern は ZK proof 外から漏れる。proof system の ZK と application-level privacy を区別する必要がある。

32. zkVM • proof-carrying execution

32.1 AIR-based zkVM

instruction decoder、register file、memory bus、program counter、syscall、range check を trace columns へ配置し、各 row の transition と global memory consistency を AIR/lookup/permutation で検査する。2018 APR はこの設計の原型である。

zkVM component	AIR/constraint	主な cost
instruction decode	opcode selector+one-hot	degree/lookup
ALU	add/mul/bitwise/custom gates	field mismatch
program counter	pc' = pc + len または branch	conditional selector
memory	address-sorted permutation/grand product	routing/lookup
range checks	bit decomposition/lookup	trace width
public I/O	boundary constraints	small
hash/precompile	special AIR/chip	dominant になりやすい

uniform VM では program table を public input として小さく保ち、実行時間だけを伸ばせる。一方、Aurora/R1CS 型は program execution を unroll するため、dynamic control flow や memory を compiler が大きな constraint system へ展開する。

32.2 Segmentation

巨大 trace を一括 proof すると peak memory が大きい。execution を segments へ分け、各 segment が state digest の入出力を持つようにし、parallel proof と recursive aggregation を行う。

$$s_{\{i+1\}} = F_i(s_i), \quad \text{Verify}(s_i, s_{\{i+1\}}) = 1, \quad \Pi = \text{Aggregate}(\pi_0, \dots, \pi_{\{k-1\}})$$

segment size は FFT efficiency、GPU occupancy、Merkle tree size、network、aggregation cost の balance で決める。小さすぎると fixed overhead が増え、大きすぎると memory spill が起きる。Figure 6 の RAM phase transition はこの設計課題を先取りしている。

32.3 Determinism と environment

proof 対象 VM は、clock、randomness、I/O、floating point、concurrency を deterministic semantics へ固定する必要がある。外部 data は commitment と signature で public statement へ bind し、oracle response の timestamp/provenance を明示する。

33. Confidential database audit

33.1 DPM を一般化する

秘密 database D と公開 commitment cmt に対し、query q 、algorithm C 、output y について

$$Com(D) = cmt \setminus \setminus wedge \setminus C(D, q) = y$$

を証明する。集合 membership/non-membership、規制 threshold、重複排除、統計集計、anomaly count、policy compliance へ拡張できる。

応用	secret witness	public statement	追加課題
医療 registry	患者 records	cohort 統計／eligibility	consent、re-identification
金融 audit	transactions/positions	solvency、risk limit	valuation oracle
supply chain	provenance events	policy 遵守	physical-digital binding
forensics	evidence database	match/non-match	chain of custody
tax/compliance	ledgers	rule satisfaction	法解釈・更新
data marketplace	raw dataset	quality metric	sampling bias

33.2 Append-only commitment

時刻 t ごとの root cmt_t を public log へ置き、update proof で $cmt_t \rightarrow cmt_{t+1}$ が authorized append/delete policy に従うことを証明する。query proof だけでなく update path も証明することで、database curator が都合の悪い record を事前に除外する余地を減らす。

$$UpdateValid(cmt_t, \Delta_t, cmt_{t+1}) = 1$$

33.3 Differential privacy との併用

ZK は input を隠しながら正しい output を証明するが、output 自体から個人情報が漏れる可能性は残る。DP mechanism $M(D; r)$ を constraint 化し、noise seed r が public beacon/commit-reveal から得られ、output が所定 privacy budget に従うことを証明できる。

ただし DP の semantic guarantee は dataset adjacency、query composition、budget accounting に依存し、ZK proof はそれらの実装が正しいことしか保証しない。

34. 科学計算・AI・検証可能 data pipeline

34.1 科学計算

simulation、optimization、numerical linear algebra、bioinformatics pipeline について、input dataset commitment、software/version hash、parameter、result digest を public statement に含める。

$$\text{Hash}(\text{code})=h_C \setminus \wedge \text{Hash}(\text{data})=h_D \setminus \wedge \text{Run}(\text{code}, \text{data}, \text{params})=\text{result}$$

有限体での exact computation は直接扱えるが、floating-point は rounding semantics、NaN、overflow、transcendental approximation を constraint 化する必要がある。fixed-point/integer arithmetic へ変換すると proof は容易になるが、数値誤差解析を別途与える。

34.2 Verifiable ML inference

model weights W を commit し、private input x または private model の下で $y=\text{NN}_W(x)$ を証明する。matrix multiplication は field-friendly だが、activation、normalization、quantization、lookup、range check が cost を決める。

component	証明しやすさ	典型的手法
integer GEMM	高	custom gate/sumcheck/matrix commitment
ReLU	中	comparison+range proof/lookup
softmax	低	polynomial/rational approximation
layer norm	低	inverse/sqrt approximation
attention	中～低	batched matrix products+approximation
quantized model	高め	small-field/lookup

ZK は model/input secrecy を提供するが、model extraction、membership inference、output leakage は application-level threat model である。また、正しい inference の証明は training data 品質、公平性、robustness を自動的に保証しない。

34.3 Verifiable training

training 全 step を証明するのは巨大で、randomness、optimizer state、data order、floating point を固定する必要がある。IVC/folding、checkpoint proof、sampled audit、proof-carrying gradient、commitment to dataset を組み合わせる研究領域である。

「所定 algorithm を実行した」ことと「良い model を得た」ことは別である。後者は統計的 generalization や causal validity の問題を含む。

34.4 ETL と data lineage

raw data→cleaning→join→aggregation→model input という pipeline の各 stage で root を更新し、proof を chain する。policy 違反 record の除外、schema validation、deduplication、access control、consent filtering を証明できる。

DPM の commitment+computation integrity を、複数 organization・複数 stage へ拡張したものと見なせる。

35. 実装・監査の設計チェックリスト

35.1 Relation と arithmetization

1. public input、witness、output、failure condition を形式的 relation として先に定義する。
2. AIR/R1CS/Plonkish/CCS/VM のどれが native computation を最小 overhead で表すか選ぶ。
3. field characteristic と native data type の相性を評価する。bitwise なら binary/small field、modular arithmetic なら対応 prime field を検討する。
4. constraint degree、trace width、cycles、lookup table、memory argument を計測する。
5. padding、domain size、vanishing set、boundary conditions に off-by-one がないか検算する。

35.2 Protocol parameters

1. target classical/quantum security を分ける。
2. field-size error、sumcheck/mixing error、LDT error、hash collision、Fiat-Shamir loss を分解する。
3. proved soundness と conjectured estimator を UI・config で明示する。
4. RS rate/blowup、folding factor、query repetitions、ZK masking degree を最適化する。
5. proof serialization を canonical 化し、domain separation tag を protocol/round/oracle ごとに設ける。

35.3 Prover engineering

1. FFT/LDE を in-place、streaming、coset-batched のどれで実装するか決める。
2. pointwise constraints と hash を SIMD/GPU へ分離し、memory traffic を profile する。
3. oracle lifetime を解析し、使用后すぐ破棄／再計算する checkpoint schedule を作る。
4. Merkle tree を row/column/coset 単位に packing し、path pruning を行う。
5. distributed proving では challenge 決定前後の dependency と network shuffle を最小化する。
6. deterministic replay と transcript logging を用意する。

35.4 Verifier and API

1. public input の長さが verifier complexity を支配しないか確認する。
2. proof parser に length bounds、canonical field encoding、duplicate path 検査を入れる。
3. batch verification で random coefficients を domain-separated に生成する。
4. verification key／program hash／AIR version を proof へ bind する。
5. failure reason を情報漏洩しない粒度で設計する。

35.5 Security testing

1. constraint omission、boundary bypass、division-by-zero、non-canonical encoding を unit test する。
2. ランダム trace corruption、Merkle path mutation、FRI layer mutation に必ず reject することを fuzz する。
3. small field で exhaustive test し、symbolic algebra で quotient divisibility を確認する。
4. Fiat-Shamir transcript の fork、reordering、cross-protocol replay を試す。
5. side-channel、memory zeroization、witness spill、crash dump、GPU shared memory を監査する。
6. third-party implementation と test vectors を相互検証する。

36. 研究課題

36.1 Proof size と authentication overhead

2018 prototype でも IOP arithmetic より Merkle 認証が支配した。STIR/WHIR、vector commitment、high-arity tree、batch opening、proof aggregation、better hash はこの層を狙う。

36.2 Linear-time prover と memory

FRI は linear でも LDE が $N \log N$ で、full oracle materialization が memory を圧迫する。Brakedown 型 code、streaming FFT、distributed FFT、hardware-aware arithmetization、incremental folding が競合・補完する。

36.3 Small fields and bitwise computation

binary towers、extension-field sumcheck、Circle domains、lookup-heavy arithmetization により、CPU/VM native word と proof field の mismatch を減らす。soundness の field-size 項を extension challenge で補う設計が重要になる。

36.4 Recursion under PQ assumptions

pairing/DLP recursion は成熟しているが非 PQ。hash-based recursion は proof verifier を AIR 化する cost が大きい。transparent preprocessing、FRI/WHIR-friendly verifier、lattice commitment、cycle-free recursion が研究対象となる。

36.5 Formal verification of proof systems

arithmetization compiler、constraint system、transcript、finite-field code、security reduction を machine-check する必要がある。特に『制約が足りない』バグは cryptographic primitive が安全でも soundness を破る。

36.6 End-to-end semantics

data provenance、legal policy、oracle authenticity、software supply chain、hardware attestation を relation へどう統合するか。DPM 例が示すように、proof system の数学的完全性と社会的 statement の妥当性の境界を明示することが不可欠である。

Appendices

記号、定理依存関係、原論文対応表、参考文献、総合評価

付録 A 記号・用語集

記号／用語	定義・本書での意味
F, F_q	有限体。原論文では主に binary field F_{2^n} 。
H	計算 trace または constraint を配置する structured domain。
L	witness RS codeword の evaluation domain。通常 H と disjoint な affine coset。
L_{cmp}	composition polynomial の evaluation domain。
$Z_S(X)$	集合 S で消える monic vanishing polynomial $\prod_{a \in S} (X-a)$ 。
$RS[F, L, \rho]$	$\text{degree} < \rho L $ の多項式を L 上で評価した Reed–Solomon code。
LDE	小 domain 上の interpolant を大 domain へ評価する low-degree extension。
$\Delta(f, C)$	word f と code C の relative Hamming distance。
T	execution steps/cycles。文脈により time bound。
S	space bound。
w	AIR state width/register 数。
d	AIR/constraint degree を 2^d で上から抑える exponent。
s or $ P $	AIR transition polynomials の数。
B	boundary constraints 集合。
AIR	Algebraic Intermediate Representation。current/next state polynomial constraints。
BAIR	binary-field/power-of-two-friendly AIR。
PAIR/BPAIR	memory permutation を伴う AIR。general computation 用。
APR	Algebraic Placement and Routing。affine graph 上の RS-encoded relation。
N	APR affine neighbor maps 集合。
Φ	APR local constraint maps 集合。
κ -independence	指定 query 数まで evaluation が一様に見える masking 性。
δ -distance	不正 witness が code から離れる下限。
Θ -overlap	composition point の neighbor 参照重複上限。
ALI	Algebraic Linking IOP。多数 oracles/constraints を二つの RPT へ batch。
RPT/IOPP	Reed–Solomon proximity testing / interactive oracle proof of proximity。
FRI	Fast Reed–Solomon IOP of Proximity。recursive folding 型 LDT。
IOP	Interactive Oracle Proof。prover message を oracle として局所 query する public-coin protocol。
STIK	Scalable Transparent IOP of Knowledge。
STARK	STIK を hash/Merkle/ROM で realize した Scalable Transparent ARgument of Knowledge。
BCS transform	IOP を random oracle model の non-interactive argument へ変換する compiler。
cmt	database/state の public commitment。
CC	communication complexity。query values+Merkle authentication paths。
APC	authentication path complexity。
λ	target soundness/security parameter。
R	RS rate $\rho=2^{-R}$ を表す integer。
k or b	zero-knowledge masking の余剰 degree/query bound parameter。
ℓ	FRI query repetition 数。
ρ_{time}	T_V/T_C 。1 未満で verification time compression。
ρ_{size}	$CC/ D $ 。1 未満で witness-size compression。

付録 B 定理・補題の依存関係

以下は主結果へ至る論理依存を、実装 pipeline と対応させたものである。

結果	入力	出力／主張	依存
Lemma B.20	NTimeSpace(T,S)	BAIR: $w \approx S$, $t \approx \log T$, $d = O(1)$	Cook-Levin 型 reduction
Theorem B.12	BAIR	APR; perfect completeness/soundness/extraction	cyclic affine embedding
Lemma B.9	NTIME(T)	BPAIR: $w = O(\log T)$	RAM trace+memory permutation
Theorem B.13	BPAIR	APR; $O(T \log T)$ routing witness	back-to-back De Bruijn
Theorem B.15	APR	two RPT instances $f^\wedge(0), g^\wedge(0)$	random linking, code distance
Theorem B.18	RS word	FRI IOPP, linear prover/log verifier	FRI analysis
Lemma B.6	BAIR + B.12/B.15/B.18	explicit-constant ZK-IOP	proved soundness
Theorem 3.4	NTimeSpace(T,S)	space-bounded ZK-STIK	B.20 + B.6
Theorem 3.5	NTIME(T)	general wi/ZK-STIK	B.9 + B.13 + B.15 + B.18
Lemma B.7	BAIR	reduced-query concrete IOP	Conj. B.17 + B.19
ZK-STARK prototype	DPM AIR	non-interactive proof system	Lemma B.7 + Merkle/Fiat-Shamir

付録 B.1 Proven／conjectured／cryptographic

層	保証	性質
AIR→APR	proved, deterministic	information-theoretic perfect completeness/soundness
ALI basic	proved	distance $\delta/(2\zeta)$, error $1/ F $
FRI basic	proved range	explicit distance-dependent error
Lemma B.6	proved	conservative query count
Lemma B.7	conjectural improvement	prototype query count
Merkle compiler	computational	collision resistance
Fiat-Shamir	ROM	non-interactivity; concrete hash instantiation is heuristic
post-quantum	plausibility	QROM/digest/security reduction に依存

付録 c 原論文の章・ページと本書の対応

原論文	page	本書
Abstract	1	0.1, 10
1 Introduction	4–13	1, 10
1.1 Main contribution	6–8	0, 10.3–10.5
1.2 Applications	8–9	31–34
1.3 Comparison	9–13	10.6, 25–30
2 Methods	14–18	2, 4–9, 11
2.1 FRI	14	7, 11.2, 19.4
2.2 AIR	14–15	4, 11.3
2.3 ALI	15–17	6, 11.4, 16
2.4 LDE	17	2, 11.5
2.5 APC/CC	17–18	9, 11.6, 23.3
3 Definitions/theorems	19–23	3, 12
Appendix A Measurements	31–32	13
Appendix B AIR→STARK	33–51	14
Appendix C APR	52–62	15
Appendix D ALI	63–70	16
Appendix E DPM AIR	71–79	17
Appendix F SHA2 AIR	80	18.1
Appendix G Routing	80–83	18.2

付録 D 原論文図の対応と読みどころ

原図	内容	本書	読みどころ
Fig.1	DPM compression rates	10/13	threshold は system ・ hardware 依存
Fig.2	theoretical ZK comparison	10	2018 時点の分類。現在は追加系統あり
Fig.3	apples-to-apples benchmark	10	security/implementation 条件差に注意
Fig.4	AIR parameters	11/18	hash choice が dominant
Fig.5	max degree reduction	11/15	本文の改善率に一桁誤記
Fig.6	prover measurements	13	RAM threshold で phase transition
Fig.7	verifier/CC	13	Merkle 認証が支配
Fig.8	AIR→APR→RPT→FRI	14	構成全体の最重要図
Fig.9	Rijndael S-box	17	binary-field friendly AIR
Fig.10	compression constraints	17	AES-DM の代数化
Fig.11	DNA data structure	17	commitment binding
Fig.12	check pairs	17	unordered allele comparison
Fig.13	matching result	17	三値 result aggregation
Fig.14	pair constraints	17	branch/equality の低次数化
Fig.15	result constraints	17	output/boundary binding

付録 E 比較表の前提

proof system 比較は、少なくとも次の条件を揃えなければならない。

- relation : AIR、R1CS、Plonkish、CCS、VM のどれか。
- instance representation : uniform program か explicit circuit か。
- security : bits、classical/quantum、ROM/QROM、knowledge assumptions。
- setup : circuit-specific、universal/updatable、transparent。
- field : prime/binary/extension、element bytes、FFT domain。
- zero knowledge : HVZK、malicious-verifier ZK、perfect/statistical/computational。
- proof of knowledge : extractor model と knowledge error。
- preprocessing : key generation、statement encoding、amortization を含むか。
- hardware : CPU/GPU/FPGA、threads、RAM、network、storage。
- measurement boundary : witness generation、serialization、commitment、proof generation、verification。

原論文 Figure 3 の価値と限界

同一 server ・ 同一 subset-sum benchmark へ複数実装を持ち込んだ点は非常に価値がある。一方、security bits、single/multi-thread、preprocessing、relation-specific optimization が完全には一致しない。結果は順位表ではなく、当時の concrete regime を示す evidence として読むべきである。

付録 F 主要参考文献

- [1] Ben-Sasson, E.; Bentov, I.; Horesh, Y.; Riabzev, M. “Scalable, transparent, and post-quantum secure computational integrity.” IACR ePrint 2018/046, 2018.
- [2] Ben-Sasson, E.; Chiesa, A.; Riabzev, M.; Spooner, N.; Virza, M.; Ward, N. P. “Aurora: Transparent Succinct Arguments for R1CS.” EUROCRYPT 2019 / IACR ePrint 2018/828.
- [3] Ben-Sasson, E.; Bentov, I.; Horesh, Y.; Riabzev, M. “Fast Reed–Solomon Interactive Oracle Proofs of Proximity.” ICALP 2018.
- [4] Ben-Sasson, E.; Chiesa, A.; Spooner, N. “Interactive Oracle Proofs.” TCC 2016.
- [5] Reingold, O.; Rothblum, G. N.; Rothblum, R. D. “Constant-Round Interactive Proofs for Delegating Computation.” STOC 2016.
- [6] Lund, C.; Fortnow, L.; Karloff, H.; Nisan, N. “Algebraic Methods for Interactive Proof Systems.” JACM 1992.
- [7] Shamir, A. “IP = PSPACE.” JACM 1992.
- [8] Kilian, J. “A Note on Efficient Zero-Knowledge Proofs and Arguments.” STOC 1992.
- [9] Micali, S. “Computationally Sound Proofs.” SIAM Journal on Computing 2000.
- [10] Merkle, R. “A Certified Digital Signature.” CRYPTO 1989.
- [11] Goldwasser, S.; Micali, S.; Rackoff, C. “The Knowledge Complexity of Interactive Proof Systems.” SIAM J. Comput. 1989.
- [12] Groth, J. “On the Size of Pairing-Based Non-interactive Arguments.” EUROCRYPT 2016 / IACR ePrint 2016/260.
- [13] Gabizon, A.; Williamson, Z. J.; Ciobotaru, O. “PLONK: Permutations over Lagrange-bases for Oecumenical Noninteractive Arguments of Knowledge.” IACR ePrint 2019/953.
- [14] Chiesa, A.; Hu, Y.; Maller, M.; Mishra, P.; Vesely, P.; Ward, N. “Marlin: Preprocessing zkSNARKs with Universal and Updatable SRS.” EUROCRYPT 2020.
- [15] Bünz, B.; Bootle, J.; Boneh, D.; Poelstra, A.; Wuille, P.; Maxwell, G. “Bulletproofs: Short Proofs for Confidential Transactions and More.” IEEE S&P 2018.
- [16] Bowe, S.; Grigg, J.; Hopwood, D. “Recursive Proof Composition without a Trusted Setup.” IACR ePrint 2019/1021.
- [17] Setty, S. “Spartan: Efficient and General-Purpose zkSNARKs without Trusted Setup.” CRYPTO 2020.
- [18] Kothapalli, A.; Setty, S.; Tzialla, I. “Nova: Recursive Zero-Knowledge Arguments from Folding Schemes.” CRYPTO 2022.
- [19] Kothapalli, A.; Setty, S. “HyperNova: Recursive Arguments for Customizable Constraint Systems.” CRYPTO 2024 extended version, IACR ePrint 2023/573.
- [20] Ames, S.; Hazay, C.; Ishai, Y.; Venkatasubramanian, M. “Ligero: Lightweight Sublinear Arguments Without a Trusted Setup.” CCS 2017.
- [21] Chiesa, A.; Ojha, D.; Spooner, N. “Fractal: Post-Quantum and Transparent Recursive Proofs from Holography.” EUROCRYPT 2020.
- [22] Golovnev, A.; Lee, J.; Setty, S.; Thaler, J.; Wahby, R. S. “Brakedown: Linear-Time and Field-Agnostic SNARKs for R1CS.” CRYPTO 2023.
- [23] Ben-Sasson, E.; Goldberg, L.; Kopparty, S.; Saraf, S. “DEEP-FRI: Sampling Outside the Box Improves Soundness.” IACR ePrint 2019/336.

- [24] Arnon, G.; Chiesa, A.; Fenzi, G.; Yogev, E. “STIR: Reed–Solomon Proximity Testing with Fewer Queries.” CRYPTO 2024 / IACR ePrint 2024/390.
- [25] Arnon, G.; Chiesa, A.; Fenzi, G.; Yogev, E. “WHIR: Reed–Solomon Proximity Testing with Super-Fast Verification.” EUROCRYPT 2025 / IACR ePrint 2024/1586.
- [26] Diamond, B. E.; Posen, J. “Succinct Arguments over Towers of Binary Fields.” EUROCRYPT 2025 / IACR ePrint 2023/1784.
- [27] Haböck, U.; Levit, D.; Papini, S. “Circle STARKs.” IACR ePrint 2024/278.
- [28] Setty, S.; Thaler, J.; Wahby, R. “Unlocking the Lookup Singularity with Lasso.” IACR ePrint 2023/1216.
- [29] Lin, S.-J.; Chung, W.-H.; Han, Y. S. “Novel Polynomial Basis and Its Application to Reed–Solomon Erasure Codes.” FOCS 2014.
- [30] Byott, N.; Chapman, R. “Power Sums over Finite Subspaces of a Field.” Finite Fields and Their Applications, 1999.
- [31] Polishchuk, A.; Spielman, D. “Nearly-Linear Size Holographic Proofs.” STOC 1994.
- [32] Ben-Sasson, E.; Sudan, M. “Short PCPs with Polylog Query Complexity.” SIAM J. Comput. 2008.
- [33] Ben-Sasson, E. et al. “Computational Integrity with a Public Random String from Quasi-Linear PCPs.” IACR ePrint 2016/646.
- [34] Fiat, A.; Shamir, A. “How to Prove Yourself: Practical Solutions to Identification and Signature Problems.” CRYPTO 1986.
- [35] Unruh, D. “Non-interactive Zero-Knowledge Proofs in the Quantum Random Oracle Model.” EUROCRYPT 2015.

付録 G 総合評価

本論文の歴史的価値は、透明・scalable・zero-knowledge・knowledge sound・plausibly post-quantum という理論的に望ましい性質を、初めて meaningful な大規模 sequential computation で具体化した点にある。

数学的には、(1) computation を AIR へ落とす、(2) register-wise RS encoding で degree を抑える、(3) ALI で多数 constraint を二つの proximity claim へ圧縮する、(4) FRI で低次数性を logarithmic query で検査する、(5) randomized encoding と self-reduction で perfect ZK を作る、という五段階が本質である。

工学的には、proof arithmetic より Merkle authentication と memory movement が支配し、field/hash/trace layout が漸近式と同等以上に重要であることを早期に示した。

Aurora は同じ基盤を R1CS へ特化し、routing を univariate sumcheck へ置き換えることで prover を $O(N \log N)$ へ、proof を大幅に短縮した。以後の DEEP-FRI、Fractal、Brakedown、STIR、WHIR、Binius、Circle STARKs は、soundness、recursion、linear prover、query 数、small fields、FFT domain という別々のボトルネックを改善している。

最終的な位置づけ

2018 ZK-STARK は現在の完成形ではない。60-bit prototype、conjectured concrete soundness、巨大 memory、比較的大きな proof という限界を持つ。それでも、AIR/IOP/FRI/Merkle という architecture を実用域へ持ち込み、後続の transparent proof systems と zkVM の設計言語を定めた基礎論文である。研究・実装の双方で読む価値は依然として高い。

— 以上 —